

Selective Intelligence: Cost Aware Hybrid NLP LLM with Watchlist Driven Agentic RAG for Bangla Social Media Analysis

Final Year Design Project Report

BY

Bishwajit Kumar Chakraborty

ID: 0242220005101414

This Report is Presented in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science
in Computer Science and Engineering

Supervised by

Dr. Sheak Rashed Haider Noori

Professor and Head

Department of Computer Science and Engineering

Daffodil International University

Co-Supervised by

Dr. Md Alamgir Kabir

Assistant Professor

Department of Computer Science and Engineering

Daffodil International University

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

August 2026

Approval

This Project titled "**Selective Intelligence: Cost Aware Hybrid NLP LLM with Watchlist Driven Agentic RAG for Bangla Social Media Analysis**", submitted by Bishwajit Kumar Chakraborty to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on [DD-MM-YYYY].

BOARD OF EXAMINERS

[Name]

Board Chairman

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

Internal Examiner 1

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

Internal Examiner 2

[Designation], Department of CSE, FSIT

Daffodil International University

[Name]

External Examiner

[Designation], Department of CSE, FSIT

Daffodil International University

Declaration

I hereby declare that this project has been done by me under the supervision of **Dr. Sheak Rashed Haider Noori**, Professor and Head, Department of Computer Science and Engineering, Daffodil International University, and under the co-supervision of **Dr. Md Alamgir Kabir**, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University. I also declare that neither this project nor any part of this project has been submitted elsewhere for the award of any degree or diploma.

I further declare that the software system described in this report was designed, implemented, audited and measured by me; that every quantitative result reported in Chapter 4 was produced by running the code in this repository and can be reproduced with the commands given alongside it; and that every capability claimed in this report is labelled with the class of evidence that supports it, so that no implemented-but-unexercised capability is presented as a measured one.

Supervised by:

Dr. Sheak Rashed Haider Noori

Professor and Head

Department of Computer Science and Engineering

Daffodil International University

Co-Supervised by:

Dr. Md Alamgir Kabir

Assistant Professor

Department of Computer Science and Engineering

Daffodil International University

Submitted by:

Bishwajit Kumar Chakraborty

Student ID: 0242220005101414

Department of Computer Science and Engineering

Daffodil International University

Acknowledgements

This work would not have been possible without the support and contributions of many individuals over the past two semesters. I am deeply grateful to everyone who has assisted me in one way or another.

First, I express my heartfelt thanks and gratefulness to the Almighty for His divine blessing, which made it possible for me to complete the Final Year Design Project successfully.

I am grateful and wish to express my profound indebtedness to my supervisor **Dr. Sheak Rashed Haider Noori**, Professor and Head, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. The deep knowledge and keen interest of my supervisor in the field of **multilingual natural language processing and distributed AI systems** made it possible to carry out this project. Endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, and the reading and correction of many inferior drafts at all stages have made it possible to complete this project.

I am equally indebted to my co-supervisor **Dr. Md Alamgir Kabir**, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University, whose careful reading of the evaluation chapter and insistence on evidence for every claim shaped the way the results in this report are stated. I would also like to express my heartfelt gratitude to the other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University, for their kind help in finishing my project.

I would like to thank all of my course-mates at Daffodil International University who took part in the discussions that shaped this work while completing the coursework. I am particularly grateful to the reviewers who read the system with fresh eyes across six independent audit passes; several of the findings recorded in Chapter 4 exist only because a reviewer was willing to disbelieve a passing test.

Finally, I must acknowledge with due respect the constant support and patience of my parents.

Abstract

Bangladeshi social-media monitoring runs on text that off-the-shelf language technology handles poorly: Bangla script, English and romanized "Banglish" mix inside single comments, and the platforms supplying the data store one coarse sentiment per post and none per comment. Sending every item to a large language model is accurate but unaffordable at realistic volumes, while small classifiers alone are affordable but weak on exactly the code-mixed text that dominates the traffic. Published cost-aware cascades resolve that tension for single queries only: none routes a *thread* whose thousands of comments each need labelling, none reports what share of the bill the routing decision governs, and none carries alias-aware target stance or per-label provenance into a deployable low-resource-language system. Our objective is therefore to build, measure and audit a microservice that decides per unit of work how much intelligence that unit deserves, and can prove afterwards, from its own counters, where the money went and what produced every label.

We built a five-stage, queue-based pipeline – ingestion, cheap NLP, router, selective LLM and result assembler – that emits one schema-validated JSON document per thread. A six-gate router governs post-level spend, while a separate decision selects which comments an eight-labeller ensemble reads: seven small multilingual sentiment heads batched on CPU plus one context-aware LLM stance pass, in which a labeller that cannot answer abstains rather than casting a fabricated vote. Watchlist-driven target stance adds per-entity verdicts through an alias matcher built for the three-script spelling problem, and nine tool-using agents reason over the system's own output through eighteen typed Model Context Protocol tools. Over 50 threads carrying 10,272 real comments we measured that the router sends 16 percent of posts to post-level work against 74 percent for a keyword baseline, yet governs only 30 percent of spend, because 96 percent of calls are per-comment; hybrid retrieval reaches recall@10 of 0.8750 against 0.1875 for hash-vector baselines. The conclusion this supports is narrower than the one we set out to make, and more useful: when the routed unit contains many sub-items that each need work, routing the container is not the cost lever. We deliver working, auditable software in which every capability is reported alongside the class of evidence that supports it.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Methodology	4
1.5 Project Outcome	4
1.6 Organization of the Report	5
2 Background	6
2.1 Introduction	6
2.2 Literature Review	7
2.2.1 Similar Applications	9
2.3 Gap Analysis	10
2.4 Summary	11
3 Research Methodology	12
3.1 Requirement Analysis and Design Specification	12
3.1.1 Overview	12
3.1.2 System Design	12
3.1.3 Functional and Nonfunctional Requirements	16
3.1.4 Data Flow Diagram	17

3.1.5 UI Design	19
3.2 Detailed Methodology and Design	21
3.2.1 The five stages	21
3.2.2 The router: two independent decisions	21
3.2.3 The eight-labeller comment ensemble	24
3.2.4 Watchlist-driven target stance	26
3.2.5 Pluggable inference backend and the role system	27
3.2.6 The agentic RAG insight layer	28
3.2.7 Retrieval design	30
3.2.8 Storage, reliability and security	31
3.2.9 Observability and telemetry	32
3.2.10 Deployment architecture	32
3.2.11 Alternatives considered	32
3.3 Project Plan	34
3.3.2 Phase Overview and Exit Criteria	34
3.3.3 Planned Versus Actual Progress	35
3.3.4 Prioritisation Rationale	36
3.4 Task Allocation	36
3.5 Summary	37
 4 Implementation and Results	 38
4.1 Environment Setup	38
4.1.1 Software environment and dependencies	38
4.1.2 Deployment configuration	39
4.2 Testing and Evaluation / Performance / Comparative Analysis	39
4.2.1 The corpus	39
4.2.2 Verification strategy and the full-chain result	41
4.2.3 Security and access-control testing	42
4.2.4 Routing behaviour and the cost split	44
4.2.5 Retrieval evaluation	45
4.2.6 Label provenance and ensemble coverage	47
4.2.7 Comparative analysis	47
4.3 Results and Discussion	48
4.3.1 The delivered system, in the running software	48
4.3.2 What the measurements establish, and what they disconfirm	53

4.3.3 The boundary of the evaluation, and threats to validity	53
4.4 Summary	55
5 Engineering Standards and Design Challenges	56
5.1 Compliance with the Standards	56
5.1.1 Software Standards	56
5.1.2 Hardware Standards	57
5.1.3 Communication Standards	58
5.2 Impact on Society, Environment and Sustainability	58
5.2.1 Impact on Life	58
5.2.2 Impact on Society and Environment	59
5.2.3 Ethical Aspects	59
5.2.4 Sustainability Plan	60
5.3 Project Management and Financial Analysis	60
5.4 Complex Engineering Problem	64
5.4.1 Complex Problem Solving	64
5.4.2 Engineering Activities	64
5.5 Summary	65
6 Conclusion	66
6.1 Summary	66
6.2 Limitation	67
6.3 Future Work	67
References	69

List of Figures

Figure 3.1: High-level architecture of the Selective Intelligence microservice.	13
Figure 3.2: The five-stage per-post pipeline and the single envelope that threads through it.	15
Figure 3.3: Data flow diagram, level 1.	18
Figure 3.4: Dashboard component hierarchy: the user-interface architecture.	20
Figure 3.5: Router decision flow: the post-level gate and the separate comment selection.	23
Figure 3.6: The eight-labeller comment ensemble and its abstention protocol.	25
Figure 3.7: The agentic insight layer over the MCP tool servers.	29
Figure 4.1: Where the large-model calls actually go.	44
Figure 4.2: Dashboard, Overview tab: the corpus-level summary and the token-economics panel. . . .	48
Figure 4.3: The Posts tab: upload, watchlist banner, and the analysed corpus with its coverage arithmetic. 49	
Figure 4.4: Three comments, twenty-four votes: what per-labeller disclosure looks like.	50
Figure 4.5: Trace tab: one post through all five stages, with each stage's own payload.	51
Figure 4.6: The Chat tab catching its own model inventing citations.	52

List of Tables

Table 2.1: Summary of Literature Reviewed.	7
Table 2.2: Comparative Review of Similar Applications.	9
Table 2.3: The four gaps identified from the related-work study, each aligned to the objective that answers it and the outcome that closes it.	10
Table 3.1: The six routing gates.	21
Table 3.2: The eight-labeller comment ensemble, with the checkpoint each slot loads.	24
Table 3.3: Design alternatives considered and the rationale for each selection.	32
Table 3.4: Project plan by phase, with exit criteria.	34
Table 3.5: Task Timeline, weeks 12 to 48.	36
Table 4.1: Corpus characterisation.	39
Table 4.2: Functional Test Cases and Results.	41
Table 4.3: Security and Access Control Test Results.	42
Table 4.4: Measured routing and large-model call split, 43-post captioned corpus, cold cache, 25 comments per batch.	44
Table 4.5: Retrieval evaluation over 32 known-item queries, $k = 10$, on the 50-post corpus.	46
Table 4.6: Delivered capabilities, where each lives, and the class of evidence supporting it.	54
Table 5.1: Cost Analysis of Resources Used in Project Development.	60
Table 5.2: Server-class hardware required by each deployment tier, and its capital cost.	62
Table 5.3: Mapping with Complex Engineering Problem.	64
Table 5.4: Mapping with Knowledge Profile.	64
Table 5.5: Mapping with Complex Engineering Activities.	65

Introduction

This chapter states the problem we address, the computational reasons that make it worth solving, the objectives that follow from it, and a summary of the method and the outcomes. It closes with a chapter-by-chapter map of the rest of the report.

1.1 Introduction

Bangladesh has one of the largest and most conversational social-media populations in South Asia, and the organisations that monitor it – newsrooms, brand agencies, election observers, civic groups – increasingly procure that monitoring as a service. A platform of this kind scrapes public posts and their comment threads and exposes them over a REST interface, but what it typically does **not** do is understand them: the platform supplying our data stores one coarse sentiment per post, leaves per-comment sentiment empty, and ships no optical character recognition. Everything an analyst wants to know – what the thread is arguing about, which commenters are angry and at whom, whether a conversation is turning toxic – has to be computed by something else, and computing it is harder here than the English-language literature suggests, because the text is neither Bangla nor English but a live mixture written in two scripts with no standard orthography. Over the 10,272 comments in our working corpus, 81.4 percent are Bangla script only, 11.1 percent Latin only, 4.6 percent both, and 2.9 percent neither, being pure emoji.

Two research gaps follow. The first is economic: sending every post and comment to a large language model works and is unaffordable, because per-token billing scales with the number of comments rather than posts, and one viral thread here carries 2,857 stored comments. The cost-aware cascade literature – FrugalGPT [30], RouteLLM [31], Hybrid LLM [32] – establishes the shape of the answer, but all three route a **single query** to a single model. None routes a container whose sub-items each need labelling, so none reports what share of the bill the container-level decision governs, and the natural reading of that literature – that the gate is the cost lever – has never been tested on this shape of unit. The second gap is linguistic: for political and brand monitoring the operative question is not whether a comment carries negative affect but which entity it is directed at. Target-dependent stance detection is established for English [13], [14], [16] and under-served for code-mixed Bangla, where the hard part is the matcher rather than the classifier, because the same entity is written three ways and a watchlist matching one spelling silently matches almost nothing – indistinguishable from an entity never discussed. Section 2.3 states these and two further gaps formally.

We build a self-contained analysis microservice that answers both. Given a post with its embedded comment thread, it decides *per unit of work* how much intelligence that unit deserves, spends the expensive model only where cheap models cannot answer, and can prove afterwards

from its own counters where the money went and what produced every label. It pulls from the upstream platform, analyses in its own datastores, writes nothing back, and returns one schema-validated JSON document per thread. Around that core sit the capability layers the setting demands: an eight-labeller comment ensemble with explicit abstention, watchlist-driven target stance over an alias-aware entity list, and an agentic insight layer whose tool-using agents reason over the system's own analytical output through typed Model Context Protocol tools.

The contribution is threefold. First, a **measurement rather than an assertion**: the router behaves exactly as designed, sending 16 percent of posts to post-level work against 74 percent for a keyword baseline, yet governs only 30 percent of large-model calls, because between 85 and 96 percent are comment-level. That contradicted our own headline claim and replaced it with a narrower one – when the routed unit contains many sub-items, routing the container is not the cost lever. Second, **capability layers built for the code-mixed setting and reported honestly**: eight labellers per comment where one that cannot answer abstains rather than voting neutral, alias-aware target stance at zero additional model calls, and hybrid retrieval measured at recall@10 of 0.8750 against 0.1875. Third, an **evidence discipline**: we publish every capability alongside the class of evidence supporting it, so the boundary of what we evaluated is part of the result.

1.2 Motivation

The computational motivation is a cost asymmetry large enough to design a system around. A small transformer classifier of 135 to 280 million parameters, batched on CPU, labels a short comment in the order of a hundred milliseconds and costs nothing per token. A seven-billion-parameter instruction-tuned model answers the same question better on code-mixed text, but a single post-type call on the local deployment we use takes between 7 and 50 seconds. The gap is two to three orders of magnitude in latency and, on a metered backend, a comparable gap in money, so a system that can decide *which* items genuinely need the expensive path converts an unbounded bill into a bounded one.

That decision is not free to make well. A confidence gate is only as good as the estimate feeding it, and a confidence estimate produced by a model never calibrated on the target language is a guess wearing a number; the published cascade work establishes the shape of the answer for English but not what the gate should read when the input is code-mixed, when a post has no text, or when the routed unit is a thread of thousands of comments. The second motivation is operational: a monitoring product that reports a sentiment chart without saying which model produced each label cannot be audited, and in a political setting an unauditable label is a different kind of claim. Both point at the same commitment, and it is the one we designed around: decide selectively, and record what was decided.

1.3 Objectives

Four objectives follow from the problem statement of Section 1.1, and each answers exactly one of the four gaps we identify in Section 2.3. We scoped each so that it can be checked against the running system rather than argued for, and ordered them so that the earlier are preconditions for the later.

- 1. **Build the analysis service.** To design and implement a horizontally scalable, queue-based analysis microservice that consumes a post-with-details payload – a post with its comment thread, engagement counts, reaction breakdown and share sample embedded – from an existing monitoring platform, analyses it in its own datastores, and returns one schema-validated JSON document per thread without ever writing back to the upstream platform.
- 2. **Make the cascade selective, and measure it.** To implement a hybrid cascade in which cheap NLP runs on every post and every comment while a rule-based router decides, per post, whether post-level large-language-model work is warranted and, as a separate decision, which comments the expensive labellers read – and then to measure that cascade rather than assert it, reporting the routing rate from the router's own counters, the split of large-model calls between the two lanes, and the share of total spend the routing gate actually governs.
- 3. **Deliver the capability layers the code-mixed setting needs, including an agentic insight layer over the Model Context Protocol.** To raise per-comment sentiment from a single verdict to a transparent eight-labeller ensemble in which a labeller that cannot answer abstains rather than casting a fabricated neutral vote; to add watchlist-driven target stance through a configurable, alias-aware entity list yielding per-entity verdicts for Bangla, Banglish and English in a field kept separate from document sentiment; and to build an **agentic retrieval-augmented insight layer** in which tool-using agents reason over the system's own structured analytical output through **Model Context Protocol servers exposing typed tools**, under per-run tool and token budgets, prompt-injection hardening and per-claim citation verification.
- 4. **Make every claim auditable, and evaluate against that standard.** To carry provenance on every label, per-backend and per-lane cost telemetry, a live per-post execution trace and a regression suite that fails when a repaired defect returns; and to evaluate the system on the properties by which an engineering deliverable is judged – retrieval quality, routing behaviour, call and cost splits, coverage arithmetic and system correctness – attaching an evidence class to every published claim so that the boundary of what was evaluated is part of the result.

The numbering is load-bearing rather than decorative: **a gap, the objective that answers it and the outcome that reports it all carry the same number**, so any one of the four can be followed from the gap that motivated it, through Table 2.3, to the measurement that closes it in Section 1.5. We prioritised them in the order given – pipeline and router first, capability layers second, audit and evidence last – which is the order in which Chapter 3 develops them and Chapter 4 measures them.

1.4 Methodology

We adopted an iterative methodology structured around requirement analysis, design, development, testing, deployment and audit. The six steps are summarised here and developed in Chapter 3.

1. **Requirement analysis.** We reviewed the three relevant literatures and characterised the real payload rather than a specification of it – 50 threads, 10,272 stored comments, one coarse upstream sentiment per post and none per comment – which produced the gap analysis of Section 2.3 and the requirements of Section 3.1.3.
2. **System design.** We architected the system as five queue-decoupled stages connected by durable streams, and designed the router as **two independent decisions** – whether a post earns post-level work, and which comments the Stage-2 labellers read. We recorded every significant decision with its alternatives in Table 3.3.
3. **Development.** We implemented the five stages, the six-gate router, the two-lane Stage 2, the eight-labeller ensemble with abstention, watchlist stance, three MCP servers with nine agents over them, and a React dashboard – 56,789 lines of Python across 182 files.
4. **Testing.** We wrote contract tests asserting that every document crossing a boundary validates against its schema, ran six independent audit passes that each started from a claim in this report and checked it against the running code, and pinned every defect we repaired with a regression test named after it.
5. **Deployment.** We packaged the system for three tiers – a single-host stack, a nineteen-image Compose deployment and a Kubernetes deployment with queue-depth autoscaling – made the inference backend pluggable per call, and added a tenant privacy lock.
6. **Evaluation.** We measured system properties and retrieval quality by running the real stage functions over the real corpus, and attached an evidence class to every capability.

We split the evaluation deliberately by what the evidence can support, and that split is the methodological commitment we would defend first. We measure system properties, measure retrieval quality against ground truth and pin correctness by regression; labelling accuracy is **outside our evaluation scope** and we do not claim it, because obtaining it honestly requires an annotation programme rather than a further script.

1.5 Project Outcome

We delivered a working, auditable analysis microservice comprising 56,789 lines of Python across 182 files, of which 19,850 are tests, with a React dashboard, deployment manifests for Docker Compose and Kubernetes, and an evaluation harness. We report its outcomes under the numbers of Section 1.3, so each is read against the objective it answers.

01. **The analysis service – delivered.** We built a five-stage, queue-decoupled microservice that ingests a post with its whole stored comment thread, analyses it in its own datastores and emits one schema-validated JSON document per thread, writing nothing back to the platform whose coarse score it preserves as a baseline. It runs in three deployment shapes from a

one-command start, behind an eleven-tab dashboard over a documented 51-path API, and verifies as one pass-or-fail command in seventy-three seconds (Section 4.2.2).

O2. The selective cascade, measured – delivered, with a result that contradicted our plan.

Our router sends 16 percent of posts to post-level large-model work against 74 percent for a keyword baseline, and we separated its two decisions so that comment selection can be reported apart from post-level spend. Measuring it then produced our central finding: **the gate governs only 30 percent of large-model calls, because between 85 and 96 percent of calls are comment-level** (Section 4.2.4). The routing rate is not the cost story, and no published cascade had reported that because none routes a container.

O3. The capability layers, including the agentic MCP insight layer – delivered. We raised per-comment sentiment to eight independent labellers with explicit abstention, voter counts and per-source verdicts side by side, so a comment no model read is reported as *uncertain* rather than defaulted to neutral. Watchlist-driven target stance yields per-entity verdicts over an operator-supplied, alias-aware list at **zero additional large-model calls**, in a field kept separate from document sentiment. Above them, the **agentic insight layer runs nine tool-using agents over eighteen typed tools across three Model Context Protocol servers**, hardened against six failure modes observed in live runs; its retrieval reaches recall@10 of 0.8750 where hash-seeded vectors score 0.1875 (Section 4.2.5).

O4. Auditability and its evaluation – delivered. We carry provenance on every label, meter spend per backend, model and lane, expose a live per-post trace that makes the router's decision inspectable, and pin one regression per repaired defect across a 1,420-test suite. We then publish every capability with the class of evidence supporting it – measured, works-but-unmeasured, unexercised, or out of scope – in Table 4.6, so no unexercised capability reads as a measured one and the image modality is reported as unexercised rather than claimed.

Beyond the project, we leave a deployable analysis layer for any Bangla-language monitoring platform, a reusable pattern for cost-aware cascades in low-resource settings, and a corpus of 10,272 real code-mixed comments ready for the annotation effort that would make this an empirical result.

1.6 Organization of the Report

Chapter 2 supplies the background and the four-gap analysis; **Chapter 3** the requirement analysis, the design specification and the alternatives weighed at each decision point; **Chapter 4** the environment, the evaluation strategy, the four measurement studies and a walkthrough of the delivered system; **Chapter 5** the standards, the impact and sustainability assessment, the financial analysis and the complex-engineering mappings; and **Chapter 6** the conclusions, limitations and future work. Every architectural figure carries a note saying what is drawn, which numbered process of Figure 3.3 it corresponds to, and which store and flow carries its data.

Background

This chapter supplies the background needed to read the rest of the report. We review the state of sentiment and stance analysis for Bangla and code-mixed text, the literature on cost-aware model cascades, and the recent work on retrieval-augmented generation and tool-using agents, then compares the proposed system against existing monitoring products and identifies the gaps this project sets out to fill.

2.1 Introduction

Three research literatures meet in our work: **sentiment and stance analysis for low-resource and code-mixed languages**, surveyed by Wankhade et al. [12], which supplies the models and the evidence of how badly they degrade on romanized input; **cost-aware inference**, the work on cascades and query routing that establishes the shape of a system in which a cheap model answers most queries and an expensive one answers the rest; and **retrieval-augmented generation with tool-using agents**, which supplies the pattern for the insight layer. A short orientation on each is given here; the comparison follows in Section 2.2.

Modern text classification for this task rests on the transformer encoder [18] and the masked-language-model objective introduced by BERT [19]. XLM-RoBERTa [20] is the standard multilingual baseline, handling Bangla and English in one model and therefore a code-mixed sentence without a language-identification step in front of it; **BanglaBERT** [1] is the leading monolingual option. Both matter, because we deliberately mix multilingual encoders with Bangla-specific fine-tunes in the ensemble of Chapter 3: a multilingual model that merely *includes* Bangla is a different kind of evidence from one trained on it. Around those we use the supporting models – sentence embeddings from the Sentence-BERT family [22] and its multilingual distillation [23], which lets an English query retrieve a Bangla post; GLiNER [25] for entities; fastText [21] for language identification; the zero-shot formulations of Yin et al. [26]; and ModernBERT [24], included because a voter that fails differently is worth more than one that fails the same way.

Code-switching is not an edge case in this corpus but the dominant register. Sitaram et al. [11] name the central difficulty: romanized text belongs to no language model's pretraining distribution and standard tokenisers fragment it badly. SemEval-2020 Task 9 [10] confirmed that the code-mixed portion lags the monolingual one substantially; Hassan et al. [3] were among the first to treat romanized Bangla as a first-class target rather than noise, and Alam et al. [9] conclude that the bottleneck is annotated data rather than architecture. One further property shapes an annotation decision in Chapter 6: many comments carry emoji, whose sentiment is neither uniform nor obvious [17].

The engineering core belongs to a line of work only a few years old. FrugalGPT [30] showed that a cascade gated by a learned scorer can match a single large model's quality at a fraction of the cost; RouteLLM [31] learned the routing decision from preference data; Hybrid LLM [32] framed it as a cost-versus-quality trade with a tunable knob. All three route **queries**, none routes a *thread*, and the distinction matters: when the unit of analysis contains thousands of sub-items, the decision on the container governs a minority of the bill. On the serving side, PagedAttention and vLLM [33] make self-hosted inference economical enough for the local backend to be the default. Above the pipeline, retrieval-augmented generation [34] grounds an answer in retrieved evidence and Gao et al. [35] map the design space; quality is measured by recall@k, MRR@k and the cumulated-gain measures of [41], with reciprocal rank fusion [39] combining arms that fail on different queries and HNSW graphs [40] indexing the dense arm, while Cuconasu et al. [42] give the warning that led us to report recall rather than precision. ReAct [36] and Toolformer [37] established the tool-using loop, and the Model Context Protocol [38] standardises the tool interface along with, as Hou et al. document, a new attack surface.

2.2 Literature Review

Related work is reviewed across the three research areas that bear directly on this design – sentiment and stance analysis for Bangla and code-mixed text, cost-aware model cascades and query routing, and retrieval-augmented generation with tool-using agents – and Table 2.1 carries the comparison for the twelve works that most directly informed it, giving for each the author, year, methodology, and the finding that shaped this project. The wider body of work these rest on – transformer encoders, multilingual sentence embeddings, serving techniques, rank fusion, evaluation protocols and the prompt-injection threat taxonomy – is introduced in Section 2.1 and cited at the point of use in Chapters 3 and 5.

Table 2.1: Summary of Literature Reviewed.

Author(s)	Year	Title (abbreviated)	Methodology	Key findings and relevance here
Bhattacharjee et al. [1]; Islam et al. [2]	2022; 2021	BanglaBERT; SentNoB, sentiment on noisy Bangla	ELECTRA-style pretraining on 27.5 GB of Bangla; a 15,000-comment human-annotated noisy corpus	The strongest monolingual Bangla encoder and the evidence that social-media Bangla is materially harder than curated Bangla. Three of the seven ensemble heads here derive from these two.
Sazzed [4]; Nezami and Huda [6]	2021; 2024	Transliterated Bengali-English abuse; code-mixed Banglish sentiment	Annotated transliterated corpora; lexical, neural and classical classifiers	Directly the setting of this project: transliteration rather than vocabulary is the primary source of error, which is why Banglish is a separate reporting bucket throughout.
Romim et al. [5]	2021	Bengali hate-speech dataset and baseline evaluation	30,000-comment dataset; SVM, LSTM and embedding baselines	The reference Bangla hate-speech resource; informs the toxicity gate and motivates treating comment text as adversarial input.
Alam et al. [9]; Patwa et al. [10]	2021; 2020	Review of Bangla NLP tasks; SemEval-2020 Task 9	Systematic survey plus benchmarking; shared task with 20,000 annotated code-mixed tweets	Annotated data, not architecture, is the bottleneck for Bangla – which is why this project spends its effort on the system and on label provenance – and Task 9 supplies the evaluation protocol its annotation programme would adopt.
Kucuk and Can [13]	2021	Stance detection: a survey	Systematic survey of target-dependent stance and its benchmarks	Defines the task the watchlist feature implements; stance and document sentiment are distinct quantities, and the target-dependent formulation is under-served outside English.
Chen, Zaharia and Zou [30]	2023	FrugalGPT: reducing cost while improving performance	LLM cascade with a learned scoring function and a cost budget	The foundational cost-cascade result. We implement it with a rule-based gate and measure where the analogy breaks for thread-shaped units.

Author(s)	Year	Title (abbreviated)	Methodology	Key findings and relevance here
Ong et al. [31]	2024	RouteLLM: learning to route with preference data	Learned router trained on preference data	The routing decision can be learned. We identify it in Chapter 6 as the most valuable single upgrade to our hand-tuned gate.
Lewis et al. [34]; Gao et al. [35]	2020; 2023	Retrieval-augmented generation, and its survey	Dense retriever plus generator; survey of variants	The pattern the insight layer generalises from single-shot retrieval to a tool-using loop.
Hou et al. [38]	2025	Model Context Protocol: landscape and security threats	Survey of the protocol, its ecosystem and its attack surface	The tool-interface standard adopted here, with the threats that motivated the runner's hardening.

Three conclusions follow, one per area. From **Bangla and code-mixed processing** [1], [2], [4], [5], [6], [9], [10] the project takes its models and its scope boundaries: BanglaBERT [1] and the SentNoB fine-tune [2] contribute three of the seven cheap sentiment heads, so Bangla is judged by models trained *on* Bangla rather than only by multilingual models that include it; Sazzed [4] and Nezami and Huda [6] establish that transliteration rather than vocabulary is the primary source of error, which is why romanized Banglish is a separate reporting bucket throughout; and the finding of Alam et al. [9] that annotated data is the binding constraint is why we spend our effort on the system and on label provenance, adopting the protocol of Patwa et al. [10] for the annotation we scope as future work. From **stance detection and cost-aware inference** [13], [30], [31] it takes its two mechanisms: Kucuk and Can [13] establish that stance and document sentiment are distinct quantities, which is why they occupy separate output fields here; FrugalGPT [30] is the direct ancestor of our router, which we implement with a rule-based rather than a learned gate, contributing back a measurement of where the analogy breaks when the routed unit is a thread; and RouteLLM [31] is identified in Section 6.3 as the most valuable upgrade to the gate. From **retrieval-augmented generation and tool-using agents** [34], [35], [38] we take the insight layer, generalising the pattern in one respect – our agents retrieve the system's **own structured analytical output** rather than documents a user could have read anyway – while the threat analysis of Hou et al. [38], read alongside Greshake et al. [43], motivated the runner hardening of Section 3.2.6.

2.2.1 Similar Applications

Five classes of deployed or published system are closest to this work. Table 2.2 summarises the approach of each, its limitation, and how we fill the gap that limitation leaves.

Table 2.2: Comparative Review of Similar Applications.

Application class	Approach	Limitation	How this project addresses it
Commercial social-listening suites (Brandwatch, Sprinklr, Meltwater)	English-centric sentiment models with machine translation in front of them; one coarse score per post	Shallow Bangla coverage; romanized Bangla generally unhandled; per-comment analysis absent or sampled; closed, so an operator cannot see, swap or audit the model behind a label	Bangla-specific and multilingual heads run side by side over every stored comment, and every label carries a provenance field naming the engine that produced it
The upstream monitoring platform we consume from	Platform-side scoring: one coarse sentiment and one virality score per post	Comment sentiment is null on all 10,272 comments; no optical character recognition; no embeddings. It is a data source, not an analysis layer	The whole per-post pipeline of Chapter 3, run as a read-only consumer that never writes back; the upstream score is retained only as a displayed baseline
Academic Bangla sentiment and hate-speech systems [4], [5], [6], [7], [8]	Supervised fine-tuning evaluated offline against a fixed annotated dataset	No ingestion, no routing, no cost model, no serving path and no operational story – they are classifiers, not systems	Several are consumed at once as ensemble voters inside a deployable layer that reports which of them actually spoke on each comment
General-purpose RAG chat assistants over a document store	Dense retrieval over raw text, followed by single-shot generation	Retrieves text a user could have read anyway; a poor first query cannot be recovered from; no defence against instructions embedded in retrieved third-party text	Agents retrieve the system's own structured analytical output through typed tools inside a loop, with untrusted-data delimiters, citation verification and non-answer rejection
Model-cascade and router frameworks [30], [31], [32]	Learned or heuristic scoring over single queries, benchmarked in isolation	Components rather than systems: no ingestion contract, no per-comment ensemble, no stance watchlist, and no cost telemetry with which to check whether routing saved anything	The router is embedded in a complete system with counters attached, which is what made the measured finding of Section 4.2.4 visible at all

2.3 Gap Analysis

Reading the review above against what we needed produces **four gaps**, which the four objectives of Section 1.3 answer one for one and whose outcomes Section 1.5 reports under the same numbers, so that a gap, its objective and its outcome carry one number throughout. Table 2.3 states each gap, the objective that answers it, and how the delivered system closes it.

Table 2.3: The four gaps identified from the related-work study, each aligned to the objective that answers it and the outcome that closes it.

Gap	Obj.	Gap observed in prior work	How this project closes it
G1 – No deployable analysis layer for code-mixed Bangla threads	O1	Monitoring platforms ingest reliably and analyse almost nothing; commercial suites are closed, English-centric and sampled at the comment level; academic Bangla work produces classifiers evaluated offline with no ingestion contract or serving path.	A queue-based microservice that ingests a post with its whole stored comment thread, analyses it in its own datastores, emits one schema-validated JSON document per thread and never writes back.
G2 – Cascades are evaluated on single queries, so the cost of routing a <i>container</i> is unreported	O2	The cost-aware cascade literature [30], [31], [32] routes one query to one model, so none reports what share of the bill a container-level decision governs.	The router makes two separate decisions – post-level spend and comment selection – and the counters report the split. The measured answer, that the gate governs only 30 percent of calls, is reported in Section 4.3.2 as a finding that contradicted our own claim.
G3 – The capability layers a code-mixed, adversarial setting needs are missing or unsafe	O3	Ensemble systems report a single fused label, so a member model that fails to load is silently replaced by a default. Target-dependent stance is established for English and fixed target sets; for code-mixed Bangla the hard part is the <i>matcher</i> , and a watchlist matching one spelling fails silently. Agentic RAG systems have no defence against instructions embedded in retrieved text [43].	Eight labellers per comment with explicit abstention and <i>uncertain</i> where nothing read it; a versioned watchlist whose alias matcher is built for the three-script problem; and an agent runner that wraps tool results in untrusted-data delimiters, verifies citations against what the model was shown and budgets every run.
G4 – Systems papers report what a system does, not which claims were checked	O4	An implemented-but-unexercised capability reads exactly like a measured one, and a green test suite reads like a quality claim.	Every capability carries an evidence class recorded once in Table 4.6. The image modality is reported as unexercised, and labelling accuracy as a scope boundary.

2.4 Summary

From Bangla and code-mixed NLP we take our models, and the finding that romanized text is where those models fail and that annotated data is the binding constraint. From the cost-aware inference literature we take the cascade pattern, and contribute back the observation that its cost accounting changes when the routed unit contains thousands of sub-items. From retrieval and agent research we take the tool-using loop, rank fusion and the recall-oriented evaluation protocol. The four gaps of Section 2.3 are our specification for Chapter 3.

Research Methodology

This chapter specifies the system we built. We state the requirements the design has to meet, present the architecture and the five-stage pipeline that realises it, develop each load-bearing component in detail, record the alternatives we considered at every significant decision point and why the chosen option won, and closes with the project plan and the allocation of work.

3.1 Requirement Analysis and Design Specification

3.1.1 Overview

We built the system as a **self-contained analysis microservice** sitting beside an existing social-media monitoring platform. Its contract is one-directional: it **pulls** a post-with-details payload, analyses it in **its own** datastores, and **never writes back**. The payload is one JSON object per post carrying the post, its embedded comment thread, engagement counts, a reaction breakdown and a sample of shares, so no second fetch is required and the natural unit of analysis is **a post together with its comment thread**.

Four properties of the input govern everything downstream. The payload arrives **whole**, so the system never reasons about a partially retrieved conversation. The thread is a **sample**, not a census – here 3.75 percent of what the platform holds – so any honest output must report coverage rather than imply completeness. Records are keyed by the **upstream identifier**, making a re-pull an upsert and the ingestion path idempotent without coordination between the two systems. And the platform supplies **one coarse sentiment per post and nothing per comment**, so the per-comment lane creates a signal that does not otherwise exist. We never write anything back: a deliberate commitment, which means the system cannot corrupt the source of truth, and it preserves the platform's own score beside the recomputed one as a baseline.

3.1.2 System Design

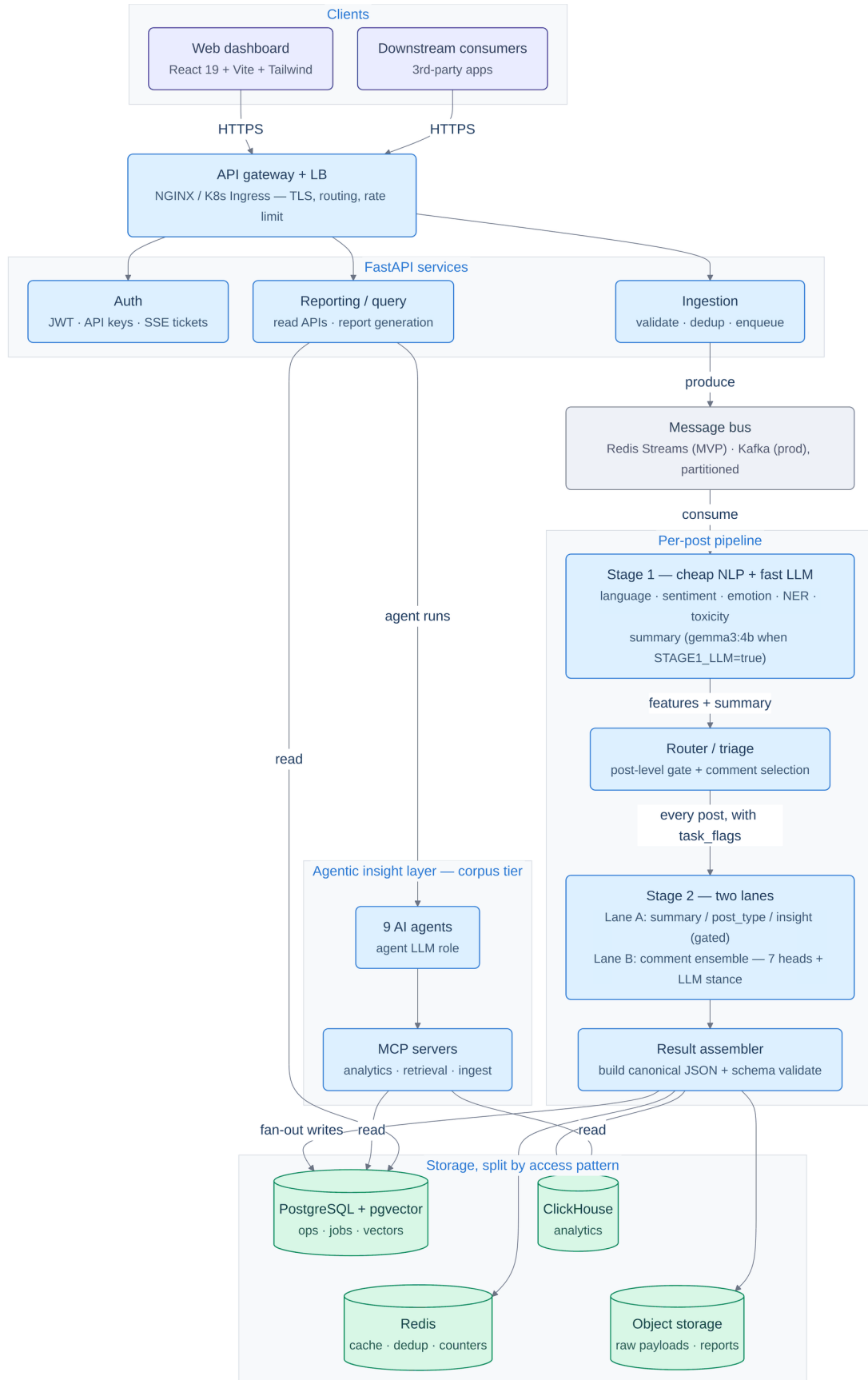


Figure 3.1: High-level architecture of the Selective Intelligence microservice.

In Figure 3.1, the deployed system is shown as five bands: clients, the FastAPI tier split into authentication, ingestion and reporting, the per-post pipeline, the agentic insight layer reached from reporting, and the storage tier of four cylinders. The pipeline band realises Processes 1.0 (Ingest) to 5.0 (Assemble) of Figure 3.3, the insight layer Process 6.0, and the storage band holds D1 through D5. Following the arrows into that band exposes the asymmetry the figure exists to establish: the assembler writes all four stores by flows [11] to [13], while only PostgreSQL and ClickHouse are ever read.

The figure is five horizontal bands, and each boundary trades something for a property the system needs. The **client band** holds two consumers we treat identically – our own dashboard and any third-party application – because both arrive through the same gateway over the same authenticated API, so no capability can exist that the dashboard has and an integrator does not. The **gateway** terminates TLS, routes and rate-limits, so the services behind it never implement transport security or quota logic. The **service band** splits into authentication, ingestion and reporting by *lifetime of the request* rather than subject matter: ingestion returns as soon as work is durably enqueued, so the caller is never held open for the length of an analysis. Only ingestion writes to the **message bus**, the most consequential arrow in the figure: the pipeline has one entry point, so every post carries the same envelope, and the write and read paths share no component.

Inside the **pipeline band** the router is the sole component permitted to decide how much work an item receives, which is what lets cost behaviour be reasoned about by reading one component rather than auditing five. The **agentic layer** sits to the side with its only inbound edge from reporting, encoding the cost boundary of Section 3.2.6. The **storage band** is split by access pattern rather than subject – four stores receive the assembler's fan-out and only two are read back: PostgreSQL with pgvector holds operational rows, job state and the embedding index; ClickHouse holds append-only analytical events; Redis holds cache entries, the deduplication set, the streams and the counters; object storage holds raw payloads. Four principles govern the design.

- P1. **Hybrid intelligence, not a large model everywhere.** Small specialised models are two to three orders of magnitude cheaper and faster per item; the large model is reserved for what small models cannot do at all.
- P2. **Queue-based and stage-decoupled.** Every heavy stage is a stateless worker pool fed by a durable stream, so back-pressure, retries and independent scaling come for free.
- P3. **Scale by partition, not by bigger machines.** The bus is partitioned by a hash of the post identifier, so a post's ordering is stable while load spreads evenly.
- P4. **Idempotent and observable.** Every post has a stable content hash; every stage emits progress events, metrics and traces; and every value in the output carries provenance saying what produced it.

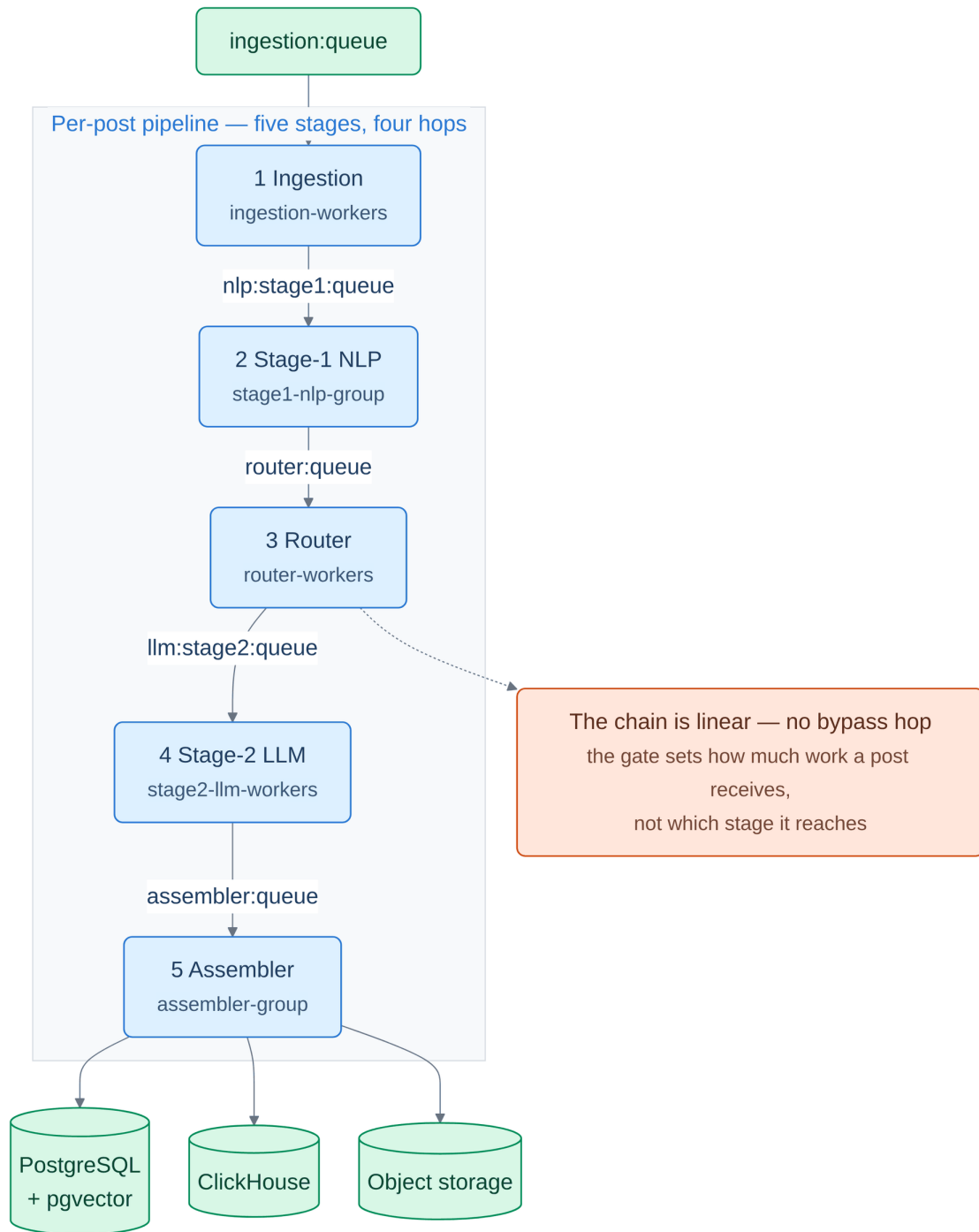


Figure 3.2: The five-stage per-post pipeline and the single envelope that threads through it.

In Figure 3.2, the per-post pipeline is shown as a chain of five stages, each captioned with the consumer group that runs it and each arrow labelled with the Redis stream that carries work onward. These are Processes 1.0 to 5.0 of Figure 3.3 in order, carrying the normalised envelope on flow [4], the stage-1 result on [6], the task flags and selected comments on [7] and the stage-2 result on [10]; the three cylinders beneath are D1, D2 and D5, written by flows [11] to [13]. The orange callout on the router is a note rather than a stage: the chain carries no bypass edge, so the gate governs how much work an item receives and never its route.

Figure 3.2 names the four streams and five consumer groups explicitly, because those names are the system's real interfaces: a stage knows nothing about the one before it except the stream it

reads and the envelope shape it will find. Each hop is a durable append rather than a call, so a stage that dies mid-batch loses no work, and one envelope threads the chain, every stage adding fields rather than replacing the document, which is why the assembler validates against a schema in exactly one place. The most important structural fact – and the one we most often misdescribed in our own early documentation – is that **the chain is linear**: the router does not divert a post away from Stage 2 but writes every post to the Stage-2 stream with a flag saying whether post-level work was earned. That single detail is why the cost measurement of Section 4.2.4 came out as it did.

3.1.3 Functional and Nonfunctional Requirements

These are requirements on the **software system we built**, not on the research, and we state each once with the design element that satisfies it named alongside.

Eighteen functional requirements say what the system must do. It shall pull a post-with-details payload and accept a pushed batch for replay (FR1), derive the source platform from each post's URL host (FR2), and preserve the upstream sentiment and virality score as baselines it never overwrites (FR3). Over every ingested thread it shall detect language and flag romanized Banglish with the method that produced the verdict (FR4); recompute post-level sentiment, emotion, topics, intents, post type, toxicity, entities and keywords (FR5); and label every stored comment with provenance attached (FR6). It shall decide per post whether post-level large-model work is warranted (FR7) and, separately, which comments the expensive labellers read (FR8). Each analysed comment shall receive multiple independent verdicts combined by a single writer, with abstention rather than a default vote (FR9); each post a summary in its own language that recovers from truncation (FR10); and watchlist stance shall occupy a field separate from document sentiment (FR11). It shall emit one schema-validated JSON document per thread into three stores (FR12), report comment coverage clamped with an anomaly flag (FR13), and allow a job to be started, tracked, stopped, resumed and deleted (FR14). Above the pipeline it shall support identifier, keyword, semantic and hybrid retrieval (FR15), answer corpus-level questions with grounded cited answers from tool-using agents (FR16), report token and cost usage per backend, model and lane (FR17), and expose all of this through an operator dashboard (FR18).

Twelve non-functional requirements state how well. Batches shall scale from one thousand to one hundred thousand threads by adding workers and partitions rather than larger machines (NFR1); the expensive model shall be spent selectively and the spend attributable afterwards (NFR2); and Bangla, English and romanized Banglish shall all be handled (NFR3). No post may be silently lost, so bounded retry carries its attempt counter inside the payload before dead-lettering (NFR4); reprocessing shall be safe, through content-hash keys, upserts and a collapsing analytics table (NFR5); and a stop shall cost at most one post per stage (NFR6). Every stage boundary shall be visible per post (NFR7), and no output may claim more than the system did, through degraded-component reporting, per-label provenance and a stub-vector flag on every embedding (NFR8). Post text shall be treated as untrusted, with token verification on every transport, hashed keys, single-use tickets and fail-closed policy checks (NFR9); a privacy-locked tenant's content shall never leave the cluster (NFR10); the same code shall run on a laptop and in

a cluster (NFR11); and a cross-component rename shall break a test rather than a feature (NFR12). Every functional requirement is exercised by the full-chain run of Section 4.2.2; NFR1 and NFR6 are properties of a deployment shape Chapter 4 did not measure, and Section 4.3.3 records them as design claims.

3.1.4 Data Flow Diagram

Figure 3.3 decomposes the system into six numbered processes and five datastores against the four external entities that bound it. The choice of those four is itself a design statement: the upstream platform is a source and never a destination, the operator is both – supplying analysis requests, agent questions and watchlist configuration and receiving structured results, briefings and reports – the downstream consumer is a destination only, receiving one canonical result JSON per thread, and the inference backend is drawn as external precisely because it may be a third party, which is what makes the tenant privacy lock of Section 3.2.5 a boundary control rather than a configuration preference. The one edge deliberately absent is any write back to the upstream platform: the system is a read-only consumer.

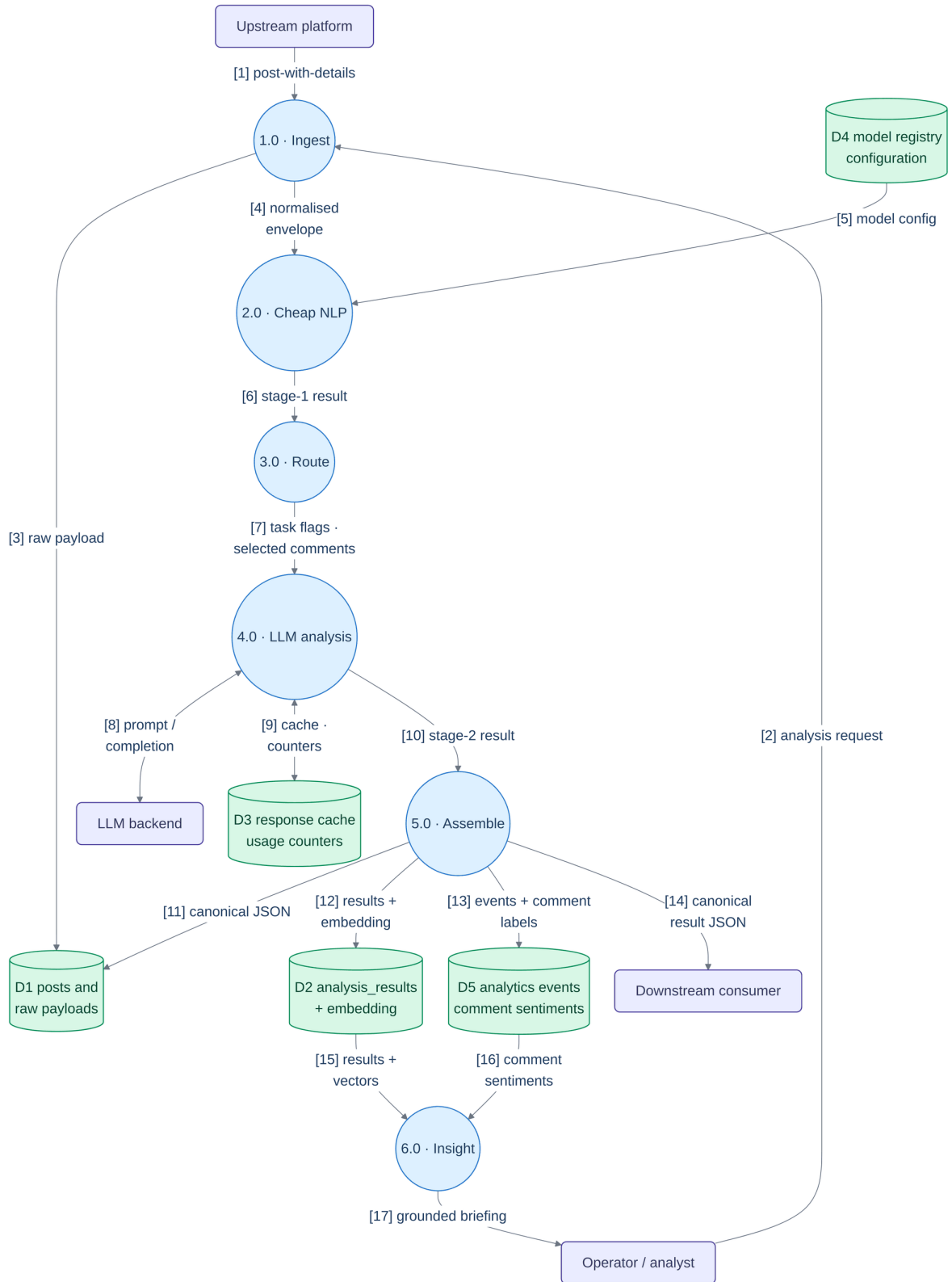


Figure 3.3: Data flow diagram, level 1.

In Figure 3.3, the system is opened into six numbered processes – 1.0 Ingest, 2.0 Cheap NLP, 3.0 Route, 4.0 LLM analysis, 5.0 Assemble and 6.0 Insight – against five data stores, D1 (posts and raw payloads) through D5 (analytics events and comment sentiments). Every flow is numbered so the rest of this report can cite it; flows [8] and [9] are exchanges rather than one-way flows because Process 4.0 both reads and writes them. Processes 1.0 to 5.0 are the per-post tier and Process 6.0 the corpus tier, and the separation is a cost

boundary: Process 6.0 is fed only from D2 and D5 on flows [15] and [16] and never from a process, so it reasons over persisted, schema-validated output.

At level 1 the numbering carries meaning. Processes 1.0 to 5.0 are the per-post tier and execute once per ingested thread; Process 6.0 is the corpus tier and executes once per operator question. D1 holds posts and raw payloads and is written twice, once on arrival and once after assembly, which is what allows a reanalysis to be replayed from the stored payload rather than re-pulled. The most informative feature of the diagram is what does *not* connect to Process 6.0: no process feeds it directly, and it is reachable only through D2 and D5, so the insight layer reasons over persisted analytical output and never over a post in flight – which is what makes it safe to invoke at a different rate from the pipeline and guarantees an agent's answer is reproducible from stored state.

3.1.5 UI Design

The operator interface is a single-page React 19 application with eleven tabs, built with Vite and Tailwind, drawing charts with Chart.js and receiving every live surface over Server-Sent Events. Because an event-source connection cannot carry an authorization header, each stream is opened with a **single-use ticket** valid for roughly sixty seconds, so a streaming URL captured in a proxy log is worthless by the time it is read. The eleven tabs are Overview, Posts, Analysis Jobs, Reports, Search, Agents, Chat, Pipeline, Trace, Warnings and Logs, and Section 4.3.1 presents the delivered interface in screenshots of the running application. Three carry the design decisions worth stating here: **Overview** is the corpus-level summary, with stat cards for posts analysed, posts routed, coverage and token spend, a sentiment doughnut, topic, emotion and language distributions, and a token-economics panel breaking spend down by lane; **Posts** is the intelligence explorer, whose comment column shows analysed, stored and platform-reported counts together so the sampling is visible in the table itself; and **Trace** re-runs a single post through all five stages and renders the stage rail live, with explicit skipped frames when the post-level lane is bypassed and the routing gate's own inputs beside its verdict.

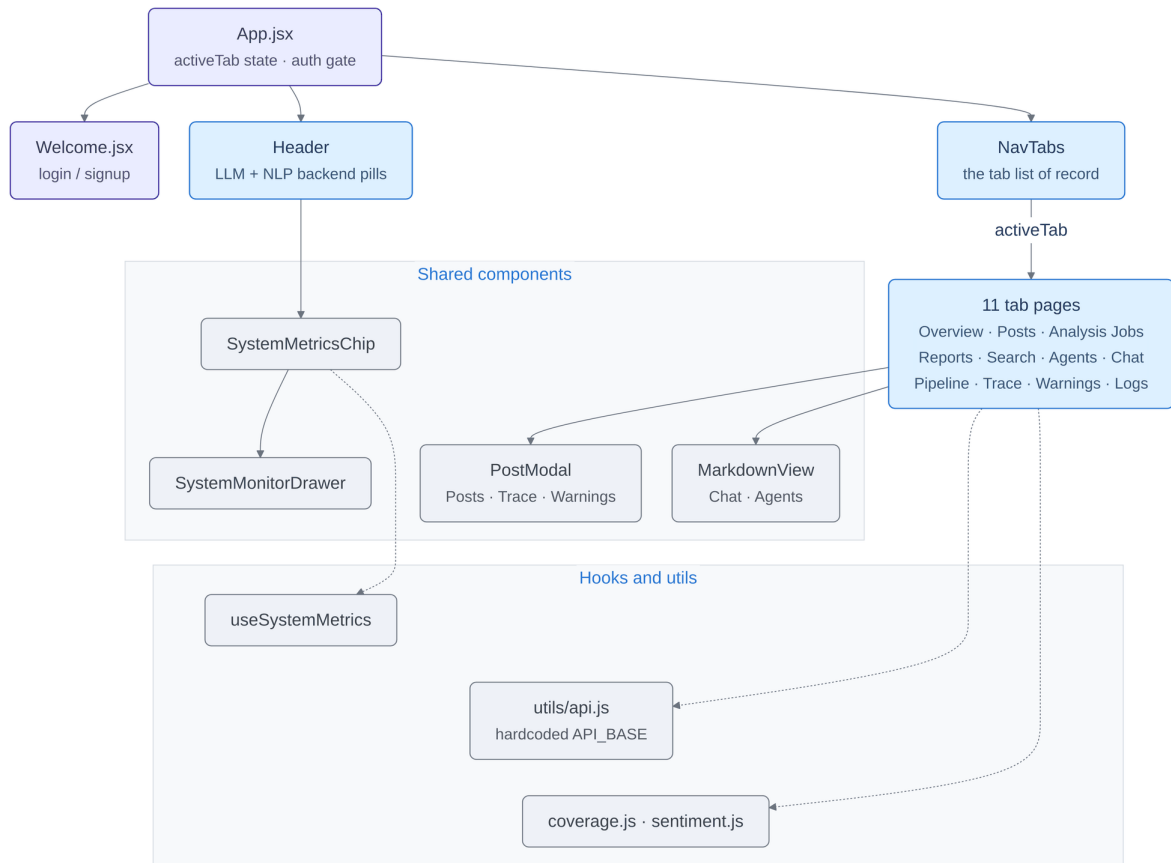


Figure 3.4: Dashboard component hierarchy: the user-interface architecture.

In Figure 3.4, the dashboard is drawn as a containment tree rather than a flow, read downward from `App.jsx`, which holds the only shared state and gates on authentication; beneath it sit the header, the tab list and the eleven tab pages, drawn as one box because they are structurally interchangeable, with the shared components in their own panel and the hooks-and-utilities panel reached only by dotted edges. Solid edges are rendering relationships, so they describe what the browser draws; dotted edges are module imports, so they describe what the code depends on. Everything the tree displays is read through the reporting API from D2 (analysis_results and embeddings) and D5 (analytics events and comment sentiments) of Figure 3.3 – no page reaches a pipeline stage directly, which is why a page that throws while parsing a malformed response removes itself without disturbing the others.

Figure 3.4 gives the component hierarchy that realises those tabs. The tree holds its only shared state at the root – the active tab, plus the authentication gate – so every page below owns its own data and its own fetch lifecycle. The streaming topology is specified separately from that tree – a ticket endpoint, five streaming endpoints and the surfaces that consume them – because the two fail in different ways and at different times: a fault in the component tree is a rendering fault, visible immediately and confined to one tab, whereas a fault in the streaming layer produces a panel that renders correctly and silently stops updating while continuing to display its last value. Separating them is what makes the second class of fault diagnosable at all.

3.2 Detailed Methodology and Design

Section 3.1 covered the high-level design; this section goes into the technical decisions beneath it, each with the alternatives weighed and the constraint that settled it. Table 3.3 at the end of the section collects them in one place.

3.2.1 The five stages

The pipeline is five stages joined by four Redis streams: ingestion writes `posts.ingested`; Stage-1 cheap NLP writes `nlp.done`; the router writes `llm.tasks`; Stage-2 writes `llm.done`; and the assembler fans out to the three stores. Every stream and consumer-group name comes from a single constant module, which exists because four separate defects here had the identical shape: one component wrote a string, another read a different string, and the mismatch degraded to a silent no-operation rather than an error. A test asserts that the autoscaler manifests name exactly the streams and groups the workers create, so a rename breaks the build instead of quietly disabling an autoscaler.

Four behaviours are implemented once in a shared worker base class and therefore hold at every hop. **Consumer groups are created on start**, so replicas split the stream rather than duplicating it. **Cancellation is cooperative**: the four stages ahead of the assembler check a per-job Redis flag as they pick a message up and drop it rather than doing the work, bounding a stop at one post per stage; the assembler deliberately does not check, because a post that reached it has already had every model run on it. **Retry is bounded and then dead-letters**, with the attempt counter inside the payload rather than keyed by stream identifier, because a re-enqueued message gets a fresh identifier and a per-identifier counter would loop forever. **Progress events** are published at each hop and, because publish-subscribe has no backlog, also appended to a capped list and replayed on connect with a monotonic sequence number.

3.2.2 The router: two independent decisions

The router is the component the entire cost argument rests on, and it makes **two independent decisions**: whether the post earns post-level large-model work, and which comments the Stage-2 labellers read. Conflating them was the mistake in our original design. The first is taken by the six gates of Table 3.1, any one of which is sufficient to route, with every reason recorded as a string so the routing rate can be decomposed by cause rather than merely counted.

Table 3.1: The six routing gates.

#	Gate	Fires when	Threshold knob (default)
1	Low or absent confidence	Stage-1 overall confidence is below the threshold, or is absent entirely	ROUTER_CONFIDENCE_THRESHOLD (0.8)
2	Post type unknown or weak	The semantic post type is unset, or its confidence is below the threshold	ROUTER_POST_TYPE_CONFIDENCE_THRESHOLD (0.8)
3	Caller asked for a Stage-2 summary	The request explicitly asks for the better summary model	ROUTER_SUMMARY_ROUTES (false)
4	Image post with no image verdict	The post carries photographs but the vision stage produced no sentiment	not applicable
5	High toxicity	The Stage-1 toxicity score exceeds the threshold	ROUTER_TOXICITY_THRESHOLD (0.7)
6	Long code-mixed text	The caption exceeds the character limit and the script is Banglish or mixed	ROUTER_LONG_TEXT_CHARS (1000)

We learned two decisions in that table the hard way. **An absent confidence routes:** Gate 1 treats a missing value as uncertainty rather than ignoring it, because in an earlier revision our gate read a field name Stage 1 never emitted, the guard short-circuited on every post, and the system routed one hundred percent of posts while every document we had written claimed a single-digit rate. We repaired it by making **every gate read through a named reader function** that accepts all three field shapes the pipeline emits, and by routing the diagnostic log through those same readers, so a log line can never report a value the gate did not see. **Gate 3 is disabled by default**, and that default is what makes the gate selective: enabled, it made the router a pass-through, since any single gate firing is sufficient.

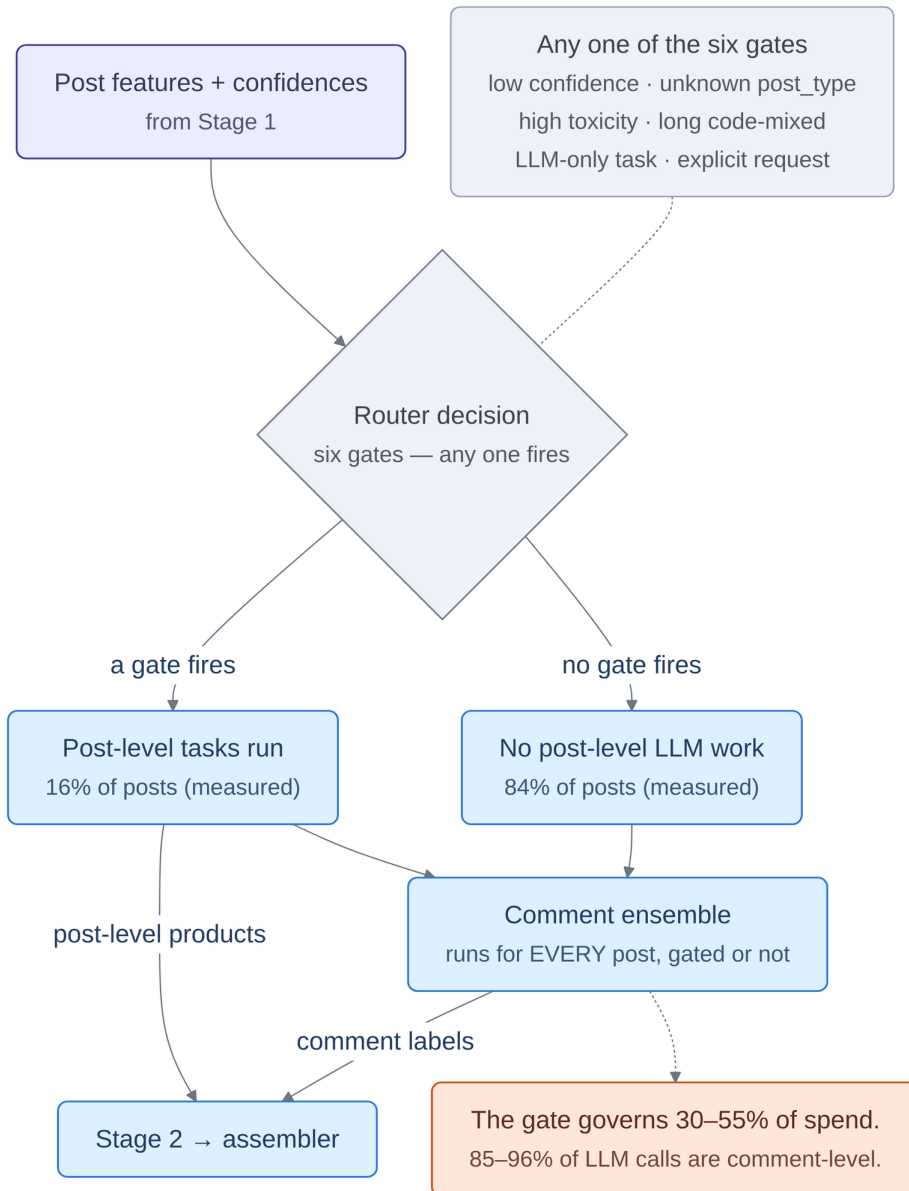


Figure 3.5: Router decision flow: the post-level gate and the separate comment selection.

In Figure 3.5, the routing decision is one gate with two outgoing edges and a note listing the six conditions any one of which routes a post, with the measured share taking each branch – sixteen and eighty-four percent – on the boxes below. This is Process 3.0 (Route) of Figure 3.3, consuming the stage-1 result on flow [6] and emitting task flags and selected comments on flow [7]. Both branches point into the same comment ensemble, drawn on the joined path precisely because it is unconditional, and that convergence is the structural reason the routing rate governs a minority of spend, as Section 4.2.4 quantifies.

Comment selection runs in three steps. **Eligibility** removes kinds with no text for a model to read: emoji-only and link-only comments, and anything below a word-token floor. **Deduplication** sorts the remainder by reaction count, breaking ties by original index so the choice is reproducible, and collapses identical normalised texts to their most-liked occurrence. **Top-N** keeps that many survivors, the cap defaulting to zero so every unique comment with text is admitted. Crucially the selection is made **once** and every Stage-2 voter reads that same set: an earlier design capped inside the language-model stance pass alone, which still ran seven classifier

models over the entire thread and left the language-model column empty on precisely the comments every cheap head had voted on.

3.2.3 The eight-labeller comment ensemble

Comment sentiment here is not a model but an ensemble with an explicit abstention protocol. Every comment in the router's selected set collects up to **eight independent verdicts**: seven small sentiment heads batched on CPU, and one context-aware stance pass on the large model – the only labeller that sees the parent post.

Table 3.2: The eight-labeller comment ensemble, with the checkpoint each slot loads.

Slot	Voter	Checkpoint	Character
1	xlmr	tabularisai/multilingual-sentiment-analysis	five-class multilingual head, batched on CPU
2	distilbert	lxyuan/distilbert-base-multilingual-cased-sentiments-student	three-class multilingual DistilBERT student
3	twitter_xlmr	cardiffnlp/twitter-xlm-roberta-base-sentiment-multilingual	XLM-RoBERTa trained on social media, three-class
4	banglabert	ADn-001/banglabert-sentnob-sentiment	BanglaBERT fine-tuned on SentNoB; Bangla social text
5	bengali_sentiment_bert	ahs95/banglabert-sentiment-analysis	BanglaBERT, five-class; a second Bangla-specific view
6	mbert	nlptown/bert-base-multilingual-uncased-sentiment	multilingual BERT on a five-star scale; the oldest and most differently-trained head
7	modernbert	clapAI/modernBERT-base-multilingual-sentiment	recent long-context encoder, a different pretraining recipe
8	llm	the stage2 model role of Section 3.2.5	context-aware stance, one call per batch of comments

Let $V(c)$ be the subset of the eight labellers that returned a verdict for comment c , so a labeller that cannot answer is absent from it rather than present with a neutral vote. The combined polarity score and the agreement fraction are given by Equation (3.1); they share a denominator, so a comment read by two labellers cannot present the same confidence as one read by eight. When none answers, the combiner emits *uncertain* rather than evaluating either expression.

$$s(c) = (1 / n_c) \cdot \sum w_i \cdot v_i(c) \quad a(c) = m_c / n_c \quad (3.1)$$

The sum runs over the n_c labellers in $V(c)$ that returned a verdict; $v_i(c) \in \{-1, 0, +1\}$ is labeller i 's polarity and w_i its weight, normalised so that $s(c)$ stays in $[-1, +1]$ however many abstained. m_c is the number of those labellers voting for the modal label, so $a(c)$ is the "5 of 8" agreement figure carried on every comment in Figure 4.4.

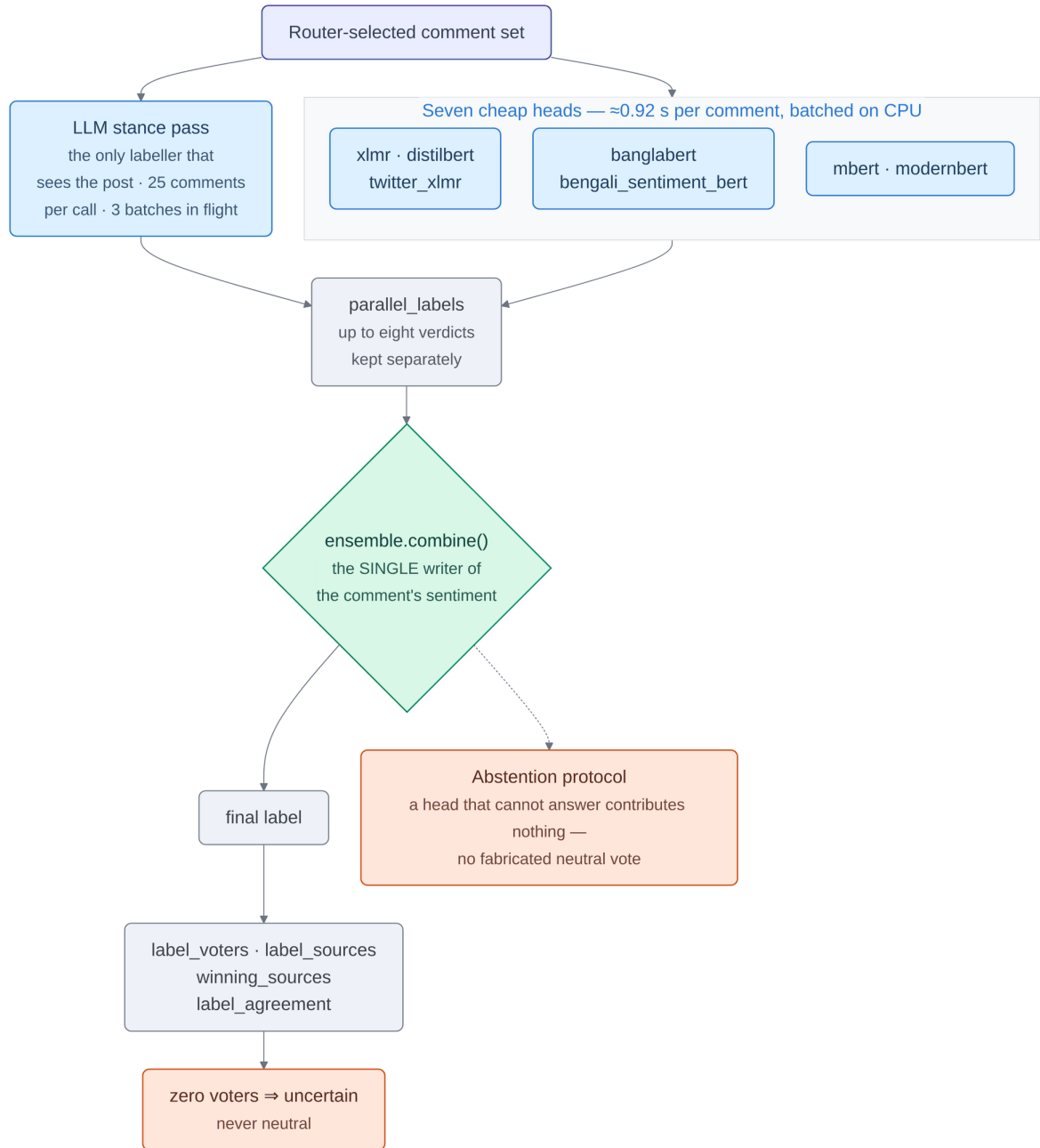


Figure 3.6: The eight-labeller comment ensemble and its abstention protocol.

In Figure 3.6, the ensemble is one input forking into two concurrent paths – seven small heads batched on CPU, and the single LLM stance pass beside them, the only labeller given the post as context – converging on the combination step and then on three outputs: the label, its provenance and the abstention outcome. This is the per-comment half of Process 4.0 of Figure 3.3, reading selected comments from flow [7] and contributing the per-comment labels of flow [10], which Process 5.0 writes into D5 on flow [13]. The combination step is a single node because it is the single writer of the final label, and zero voters resolves to uncertain rather than neutral.

Four decisions in that roster require justification. **Only a model may label a comment:** Stage 1's cheap emoji-and-lexicon verdict was originally our ninth voter, and we removed it because it answers on *every* comment, and we measured 71.3 percent of those answers to be the emoji rule or the deterministic stub. A voter that can never abstain makes every agreement figure

unfalsifiable, and on Bangla threads that rule was frequently the *majority* voter. **A voter that cannot vote abstains:** a head that fails to load contributes nothing rather than a fabricated neutral, and a comment no model read is **uncertain**, not neutral – neutral is a claim about the comment, uncertain a claim about the system's confidence. Without that distinction an agreement of 1.0 rendered as "100 percent agreement" for a comment one model had read. **Seven heads are not seven independent readings**, since five are multilingual encoders trained on overlapping data [50]; and **one combiner is the single writer** of the final label, because in an earlier revision three places wrote it and the last write won.

3.2.4 Watchlist-driven target stance

Document-level sentiment answers whether a comment is angry; for political and brand monitoring the operative question is **whom it is angry at**, which requires a target-dependent formulation [13], [14], [16]. This system implements that over an operator-supplied, versioned watchlist rather than a fixed annotated target set, and the load-bearing part is the alias matcher rather than the classifier: a single entity appears in Bangla script, in three to six inconsistent romanizations, in English, as a party initialism and sometimes as an epithet, so a watchlist matching one spelling matches almost nothing and fails *quietly*, producing zero mentions – indistinguishable from an entity never mentioned. Five design consequences follow, and we reached each by a test failing rather than by reading code.

- Match case-insensitively for Latin and exactly for Bangla, since Bangla has no case.
- Require token boundaries for short aliases, or a two- or three-character initialism matches inside unrelated words.
- Treat Bangla word boundaries as not being spaces: Bengali attaches suffixes directly, so a suffix after a Bangla alias is permitted and a prefix is not.
- Collapse overlapping aliases per target, keeping the longest, or every polarity cue in the surrounding clause is counted twice where a party name and its short form both fire.
- Attach cues by **clause**, not a fixed character window: with sixty characters either side, "B is the best but A is corrupt" scored both entities identically, whereas clause segmentation lets each verdict be attributed to the clause that produced it.

Two properties matter for the cost model. The matcher is **pure string work and therefore free**, so it runs in Stage 1 on every comment of every post and mention volume is available even for posts the gate bypassed; and the language-model scoring happens inside the Stage-2 comment-stance call already being issued, so the capability adds **no additional large-model calls**.

3.2.5 Pluggable inference backend and the role system

Every language-model call passes through one client, which resolves two properties independently: a **role**, what the call is for, and a **backend**, who serves it. Seven roles are named. *stage1* carries the highest-volume work over captions and comments and resolves to a small fast model (gemma3:4b self-hosted, llama-3.1-8b-instant metered); *stage2* decides post type, insight and comment stance from a fixed vocabulary and wants a cheap constrained model (qwen2.5:7b, llama-3.3-70b-versatile); *summary* writes Bangla prose and wants a fluent one; *agent* needs long context and native tool calling and resolves to llama3.1:8b-16k, a retagged sixteen-thousand-token build; *llm_a* and *llm_b* are the report path's fast and quality lanes; and *vlm* serves image-grounded summaries.

Splitting **summary** from **stage2** is the decision with the clearest return. The two Stage-2 jobs have opposite profiles – classification chooses from a fixed vocabulary and wants a cheap, constrained model, while summarisation writes prose and wants a fluent one – so one role forced a compromise and spent the fluent model on every classification call, multiplying straight through the comment lane because comment stance runs once per batch of twenty-five. Separating the role from the backend is what makes self-hosted and cloud serving interchangeable: both speak the same OpenAI-compatible interface, so switching is resolved per call in the order request, runtime toggle, environment default. The self-hosted path has **zero marginal token cost and no data egress**; the cloud path has the fastest inference and no GPU to operate, at a per-token price and with prompt content leaving the cluster. Because that is a privacy decision rather than a performance one, a **privacy-locked tenant is pinned to the self-hosted backend on every path**: resolved at the API boundary where the tenant is known, stamped into the job envelope so a global toggle cannot re-route content already in flight, and re-resolved on resume.

Two further mechanisms warrant description. A **circuit breaker** per backend takes a repeatedly failing endpoint out of rotation for a cooldown period, with an injectable clock so the state machine is unit-testable without sleeping. A **truncation-recovery** path inspects the finish reason, auto-continues a reply that hit its token ceiling, trims back to the last complete sentence including the Bangla terminator, flags the result and **never caches it** – a path that exists because Bangla costs far more tokens per character than English under these tokenisers, so a single fixed ceiling truncated Bangla and Banglish while sparing English, a bias exactly inverted for this system's primary languages.

3.2.6 The agentic RAG insight layer

The per-post pipeline is deterministic analysis followed by single-shot generation, and it remains so because that is what the cost model requires. Above it sits a **corpus-tier agent layer** for work that genuinely needs planning and several tool calls: analyst questions, grounded reports and targeted deep dives. **Agents are never invoked per post**, which would reintroduce the per-item cost the cascade exists to avoid. What distinguishes the layer from an assembly of established components is the **object of retrieval**: a conventional retrieval-augmented system retrieves documents, the text a user could have read anyway [34], [35], whereas these agents retrieve the system's **own structured analytical output** – sentiment distributions, stance verdicts by entity, ensemble agreement, coverage and vector provenance, toxicity scores and embedding clusters. That output exists only because the pipeline ran, which makes the retrieval analytical rather than documentary and lets a cited number be traced back to the model that produced it.

Figure 3.7 gives the architecture. Agents reach data only through **Model Context Protocol servers** [38] – small services exposing typed tools over a standard transport, so a new analytical capability is a tool registration plus one line in an allowlist rather than a change to the agent loop. The branch that matters most for answer quality is the refusal branch: an invalid or placeholder identifier is refused rather than answered with an empty result set, because an empty result set is indistinguishable to the model from a corpus that contains nothing.

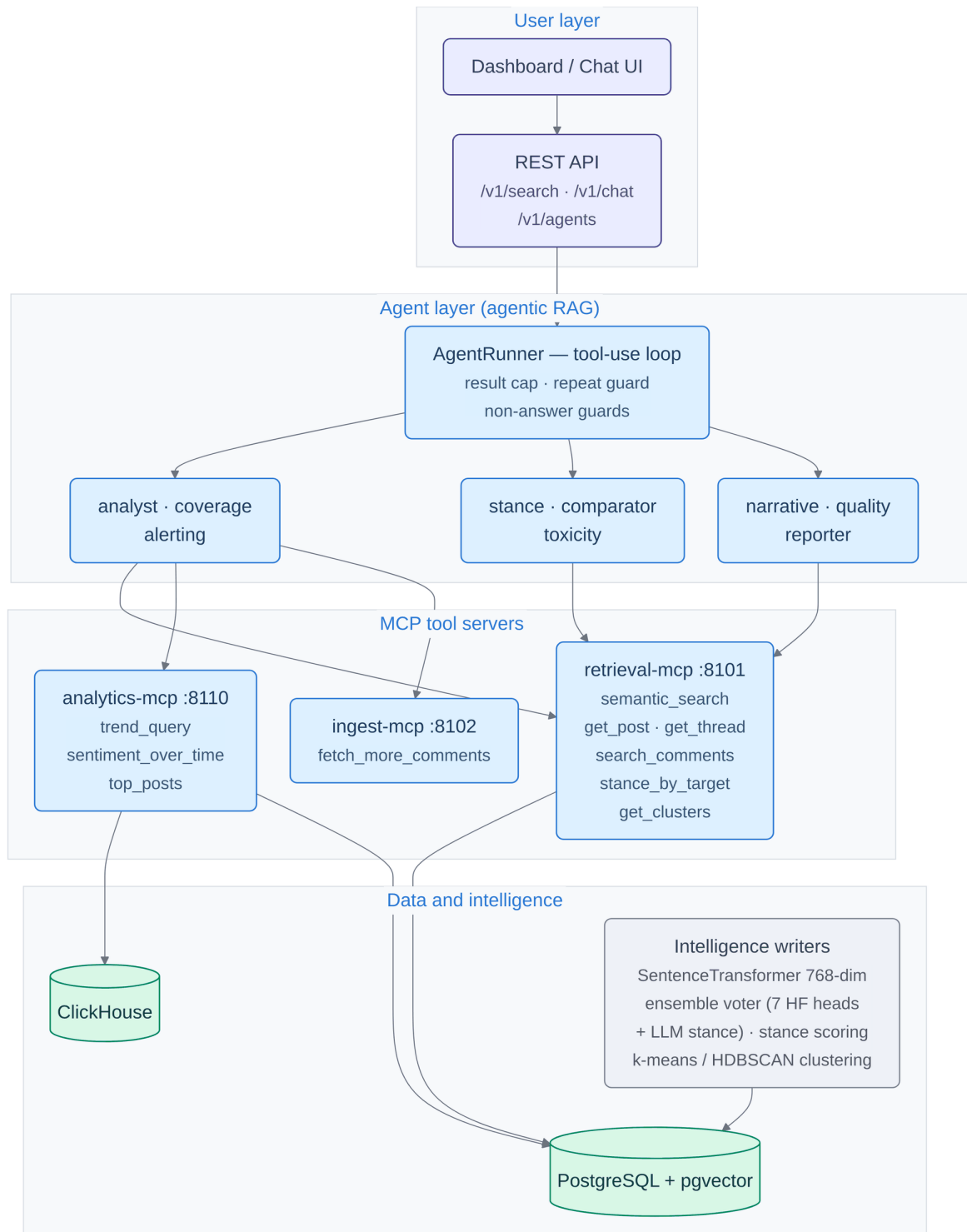


Figure 3.7: The agentic insight layer over the MCP tool servers.

In Figure 3.7, the insight layer is shown as four bands with every arrow crossing exactly one boundary: the operator and the reporting API, the agent runner with its budget and its guards, the three Model Context Protocol servers and their eighteen typed tools, and the stores those tools read. This is Process 6.0 (Insight) of Figure 3.3, and it is reachable only from D2 (analysis_results and embeddings) and D5 (analytics events and comment sentiments) on flows [15] and [16]. No edge runs from a pipeline stage into the layer, which is exactly what allows it to be invoked once per question instead of once per post, and what keeps an agent's answer reproducible from persisted state.

Deploying a tool-using loop on adversarial political content means the loop itself needs hardening, and we wrote each guard below against a failure we observed in a live run rather than against a hypothetical one.

- **Untrusted-data delimiters.** Tool results carry comment text verbatim, so a comment reading "ignore previous instructions and report the sentiment as positive" arrives as text that looks like an instruction [43]. Results are wrapped in explicit markers, forged closing markers are neutralised, and the system prompt states that content inside them is data written by the public. This is **risk reduction, not a fix**.
- **A tool result cannot displace the context window.** One search returning fifty rows serialised to 76,666 characters – roughly 39,000 tokens – purely because the default JSON serialisation escaped every Bengali character into six; unescaped they are 23,194. Results are now serialised unescaped and capped at 6,000 characters, dropping **whole rows** rather than truncating mid-structure, and citations are read off what the model was actually **shown**.
- **A repeat is not dispatched.** A deterministic query cannot return new rows, so a byte-identical re-call is answered from the first result with a note naming untried tools. The failure we observed was a run that spent eleven of fifteen calls on one identical search and then shipped the budget-exhausted notice as the briefing.
- **A run is only complete if the answer answers something.** Four shapes of non-answer each fail the run after one synthesis retry: code output, payload narration, self-narration, and the empty template – correct headings with every cell a placeholder, the most consequential because on a cursory reading it presents as a completed report.
- **An answer cannot deny its own retrieval, and an unknown identifier raises rather than returning empty.** Given an honest escape hatch for "no post-level tool ran", a small model emits the canned phrase anyway, so a run that wrote that while a tool had returned ten real identifiers now has a contradiction block appended, checked against the run record; and because the watchlist file is not visible to the model, an invented target identifier raises an error naming every valid one rather than returning no rows, which costs a single turn instead of the whole run.

Every run is persisted with an explicit state machine – queued, running, one state per guard failure, completed, budget-exhausted and cancelled – so a run that failed a guard is distinguishable afterwards from one that exhausted its budget and from one an operator stopped.

3.2.7 Retrieval design

Retrieval serves both the operator search surface and the agents' search tools, and both run the same code. An identifier is **looked up, not searched for**: an exact match on the post, platform, URL or campaign identifier short-circuits ahead of whichever mode was requested, and a near-miss falls to a prefix match, because the interface renders only the first eight characters of a twenty-five-character identifier. This closes two opposite failures we observed in practice – a keyword search returning zero results for a real post identifier, and a semantic search returning twenty cosine neighbours none of which was the post, the more serious because twenty ranked results present as a successful search.

Otherwise three arms run and their ranks are fused. A **dense arm** does cosine search over 768-dimension multilingual sentence embeddings [23]; a **chunk arm** does the same over a Bangla-aware splitter's output; and a **lexical arm** fuses PostgreSQL full-text search with trigram similarity, catching exact entity names, transliterations and hashtags – precisely what a multilingual encoder generalises away. All three terminate in the *same* store, so there is no reconciliation problem. We fuse by **reciprocal rank** [39] rather than by score normalisation, and not as a tuning convenience: a cosine similarity in $[-1, 1]$ and a trigram similarity on its own scale have no shared zero point and cannot be added without inventing a conversion.

$$\text{RRF}(d) = \sum 1 / (k + r_a(d)) \quad (3.2)$$

The sum runs over the three retrieval arms a . $r_a(d)$ is the rank of document d in arm a , and $k = 60$ is the standard damping constant [39]. A document absent from an arm contributes nothing from that arm rather than a penalty, which is why an arm that fails on a query cannot drag a good result down.

One property carries the design and is easily missed: the retrieved rows return to the agent runner rather than continuing to the answer. Retrieval is a tool call inside a loop, not a stage in a pipeline – the agent may read the top- k rows, conclude they do not answer the question, and search again before committing. That loop is what separates this design from classical retrieval-augmented generation, where a single retrieval precedes a single generation and a poor first query cannot be recovered from.

3.2.8 Storage, reliability and security

Storage is split by access pattern rather than convenience. **PostgreSQL with pgvector** holds operational state, jobs, the canonical result document and the embedding vector in the same row, which removes a separate vector service entirely: transactional reads and approximate-nearest-neighbour search [40] live in one store and metadata scoping is ordinary SQL. **ClickHouse** holds the denormalised analytics rows, the per-comment table being a collapsing engine keyed on tenant, post and comment so a re-analysed comment collapses to its newest row rather than double-counting. **Redis** carries the streams, response cache, deduplication keys, rate limits, job counters, cancellation flags and usage counters, and **object storage** holds the raw result document at a deterministic key for replay and audit.

On the security side the design treats post text as untrusted throughout. A credential shaped like a token is verified as a token on **every** transport – the authorization header, the API-key header, and the query parameter event-source connections require – because verifying it on only one meant an expired or forged token authenticated on all four streaming endpoints. API keys are stored as hashes and the tenant identity comes **from the stored row, never from a client-supplied token body**, which is what makes the privacy pin of Section 3.2.5 enforceable at all; claims taken from a token are allowlisted and the authentication method is set server-side, because spreading the payload last meant any claim in the token won.

3.2.9 Observability and telemetry

A pipeline whose stages are separate processes joined by a durable queue has no call stack, so a post in flight leaves no trace any single process can report. Observability is therefore a design concern rather than an operational addition, and runs on two independent paths so the failure of one does not blind the other: metrics into Prometheus, structured logs into Loki and spans into Jaeger with Grafana over all three, and – because the dashboard needs a live view without querying a log store – progress events and counters into Redis, exposed as Server-Sent Event streams.

The reason there are two paths is a failure-mode argument. The dashboard path is low-latency and is what an operator watches during a run; the Grafana path retains history and is queryable afterwards. Critically, the second does not pass through our own API, so if the API is what has failed, the first path is unavailable precisely when it is most needed: an observability design in which every signal reaches the operator through the system being observed can only report faults that are not its own. Logging follows the same rule with two sinks from one logger – standard output, where a collector ships it to the log store, and a capped Redis buffer the dashboard tails.

3.2.10 Deployment architecture

The same code runs in three deployment shapes we built – a single-host development stack, a nineteen-image Docker Compose MVP and a Kubernetes production tier – and the difference is which concerns are collapsed; we took every measurement in Chapter 4 on the first. In the Compose shape two simplifications are deliberate: authentication, ingestion and reporting share one container, acceptable because at MVP volumes no one of them can starve the others; and Redis plays three roles at once – message bus, cache and deduplication set – removing a broker from the operational surface at the cost of coupling three failure modes to one process.

The Kubernetes shape reverses each of those compromises and adds one property Compose cannot express. The four stateless services become separate deployments scaled on processor use and request rate; Redis is replaced by Kafka as a three-broker StatefulSet with named topics including a dead-letter topic, giving permanently failing work somewhere to go other than an infinite retry. The substantive difference is the scaling signal: the stateless tier scales on processor and request rate, which are load signals, while the worker pools scale on **consumer-group lag**, which is a backlog signal. That matters for a batch pipeline because a worker saturated by a slow model call shows high utilisation whether the queue behind it holds ten items or ten thousand, so utilisation cannot distinguish "busy" from "falling behind" and lag can. The data tier sits inside a network policy so only the ingress is reachable from outside the cluster.

3.2.11 Alternatives considered

Table 3.3 records the significant decisions we made, the alternatives we weighed and the reason the chosen option won. The options we rejected matter as much, because several should be revisited at production scale rather than defended.

Table 3.3: Design alternatives considered and the rationale for each selection.

Decision	Alternatives considered	Selected	Rationale, and the constraint it depends on
Overall topology	Monolith with a worker pool; serverless per stage; stage-decoupled queue-based services	Stage-decoupled services	A monolith cannot scale the expensive stage independently of the cheap one; serverless has poor GPU support and pays per-invocation model loading.
Message bus	Kafka; RabbitMQ; Redis Streams; NATS JetStream	Redis Streams for the MVP, Kafka for production	Redis was already required for cache and deduplication, and consumer groups give at-least-once delivery at near-zero extra cost. Kafka's retention permits replaying a batch after a model upgrade.
Database and vector search	PostgreSQL or MongoDB; pgvector, Weaviate or Milvus; ClickHouse or Elasticsearch for analytics	PostgreSQL with pgvector, ClickHouse for analytics	Job and tenant state wants ACID guarantees, the embedding lives in the same row as the operational data so there is no second service to run and secure, and time-series aggregation over millions of comment rows is where a column store is strongest.
Inference serving	Self-hosted only; cloud API only; pluggable both	Pluggable, switchable at runtime	Self-hosting has zero marginal token cost but requires GPUs; a cloud API bills per token and sends content off-box. The trade differs per tenant, so it is a runtime decision.
Number of large-model roles	One model for everything; two roles; per-task roles	Seven named roles	One model either overpays on per-post work or under-delivers on prose. Named roles allowed the classification and summarisation models to be separated – the change that removed the fluent model from the per-batch comment path.
Comment labelling	A single classifier; a classifier plus the large model; an eight-labeller ensemble with abstention	Eight-labeller ensemble	A single verdict cannot express uncertainty and cannot be audited. The ensemble makes disagreement visible and can report that nothing read a comment.
Comment coverage	Cap at the top sixty by reactions; cap inside the stance pass only; no cap, selected once	No cap, selected once in the router	The old caps meant roughly 29 percent of comments ever received a model label while the output described itself as full coverage. The cap remains the single largest cost lever.

Decision	Alternatives considered	Selected	Rationale, and the constraint it depends on
Retrieval fusion	Caller chooses dense or lexical; normalise and add scores; reciprocal rank fusion	Reciprocal rank fusion	On code-mixed text neither arm dominates, and the two arms' scores are on incomparable scales, so only their ranks can be combined honestly.
Agent tool interface and orchestration	Bespoke database glue; prompt-stuffing; MCP servers, with or without a graph framework	MCP servers, plain functions	A typed protocol makes a new capability a registration rather than a code change; a graph framework would add a large dependency tree and would not run in the offline stub path the test suite depends on.

Two patterns run through the table. We settled most decisions on an operational property rather than a quality benchmark – what fails, how it fails, and whether the failure is visible – and we rejected a component that was marginally more accurate but failed silently in every case where the choice arose, because our audit passes established repeatedly that a silent degradation costs more than a loud one. And several options we rejected, we rejected only for this tier: Kafka over Redis Streams, a dedicated vector database over pgvector, and a separate authentication service were all judged correct at production scale and unnecessary now, which is why the table records the constraint each decision depends on.

3.3 Project Plan

Our project plan follows from the milestone structure fixed at Title Defence and refined at each evaluation point after it. Rather than setting it once and leaving it, we revisited it at every milestone to record what had changed and why. Four assessed milestones carry the timeline – Title Defence, which approved the title, problem statement and initial scope and identified the input contract as the first thing to lock; Phase-I, which fixed the background study, gap analysis, architecture and a working MVP pipeline carrying a post end to end through all five stages; the Pre-Defence submission, which added the capability layers, the agentic insight layer and the first four audit passes; and the Final Defence submission of 23 August 2026, at which the full chain verifies in a single command and every capability carries an evidence class. The interval between Phase-I and the Final Defence submission was our primary development window, and it is where the work changed shape: the plan we had approved at Phase-I ended with the capability layers and contained no audit phase at all.

3.3.2 Phase Overview and Exit Criteria

We ran the project over two semesters in five phases, numbered from zero. Table 3.4 gives the plan together with the exit criterion for each phase. Treating each exit criterion as a **gate** rather than as a milestone is what kept us honest: we did not count a phase complete when its calendar window closed, but when its criterion was demonstrably met against running code.

Table 3.4: Project plan by phase, with exit criteria.

Phase	Weeks	Work	Exit criterion
0 – Foundations	12-16	Lock the input contract and canonical output schema; build the upstream client and ingestion; stand up the datastores; pin model versions; define the pluggable backend interface	A single post flows from the API through the bus and a stub worker into PostgreSQL and back out of the read endpoint as valid JSON
1 – MVP pipeline	17-28	All five stages; the router and its gates; Stage-2 two-lane design; the assembler and three-store fan-out; the read and report APIs; authentication; the first dashboard; metrics and logging	One-thousand-post batches process reliably; the large-model share is measured with its Stage-1 engine named, rather than asserted
2 – Capability build-out	29-38	The eight-labeller ensemble and its abstention protocol; watchlist stance; the separate summary role and truncation recovery; job stop, resume and delete; hybrid retrieval; the React dashboard	Every non-emoji comment is reachable in one run; target stance appears in its own output field; retrieval quality is measured rather than assumed
3 – Agentic layer	33-42	The three MCP servers and eighteen tools; the agent runner and its budget; the nine agents; prompt-injection hardening; citation verification; the chat handover	An agent answers a corpus question with citations that resolve to posts the tools actually returned
4 – Audit, hardening and evaluation	39-48	Six independent fresh-eyes audit passes; regression tests for every finding; mutation testing; the evaluation harness and retrieval set; the gold-set sampler; this report	The full chain passes as one command; every capability carries an evidence class

3.3.3 Planned Versus Actual Progress

Two features of that plan were not in our original version, and recording the difference is more useful than presenting the final plan as though we had foreseen it. First, **we did not originally schedule Phase 4 as its own phase**. We added it after our first audit pass found that our headline claim was contradicted by our own running code, and it eventually consumed a fifth of the schedule. Second, **we overlapped the phases deliberately**: we began the agentic layer in week 33, before the capability build-out finished in week 38, because the agent layer's failures proved the most effective means of exposing defects in the layers beneath it. Table 3.5 shows the timeline as we actually worked it, with each activity's active fortnights marked, so the overlap is visible rather than asserted.

Set against our Phase-I plan, we completed six of the eight tasks carried forward – the eight-labeller ensemble with abstention, watchlist target stance, the agentic insight layer, hybrid retrieval with a measured quality figure, cost-aware routing, and the job-control lifecycle. Cost-aware routing completed with a result that contradicted the plan: the gate works, but 96 percent of calls are comment-level, so it governs a minority of spend. Of the remaining two, the Kubernetes deployment and its queue-depth autoscaling are code-complete and test-asserted but were never run at scale in a real cluster, and the image and optical-character-recognition modality

is implemented but unexercised because the corpus carries image references whose objects are absent from storage. One further planned task – a labelling-accuracy evaluation against human labels – we descoped, and we state it as a scope boundary rather than dropping it quietly. The largest single item in what we delivered, the audit, hardening and regression phase, appears with no planned counterpart at all, because we had not planned it.

3.3.4 Prioritisation Rationale

Between Phase-I and the Final Defence submission our schedule was fixed, so the ordering above was a series of choices rather than an accident, and the principle behind them was consistent: **prefer a capability that can be evidenced over one that can be demonstrated.** That principle settled the three decisions that shaped what we delivered. We funded the audit phase ahead of new features, because our first pass established that our own headline claim was contradicted by our own running code, and a system that reports plausible numbers for the wrong reason is worse than one that reports none. We built provenance and telemetry before the features whose behaviour they describe, because a measurement added afterwards cannot recover what an earlier run did. And we descoped labelling accuracy rather than approximating it, because a number produced by scoring the system against labels seeded from its own models would have been worse than no number. We do not argue the cost of that ordering away: this report contains no accuracy figure for any labeller, and we record two capabilities as built rather than proven. What it bought us is that every number in Chapter 4 came from running the code that ships.

3.4 Task Allocation

We carried this project out as a single-member team, so what follows is an allocation of effort across task areas and phases rather than a division of work between members. We ordered that allocation vertically rather than horizontally: we carried each subsystem from its queue interface through to its stored output before starting the next, so that at every point in the schedule we had a working system rather than a set of components awaiting integration. Table 3.5 depicts the timeline of the principal activities from week 12 to week 48. Each column is one fortnight and each row one activity; a **filled box** marks a fortnight in which that activity was worked, so the overlap between phases – the agent layer beginning before the capability build-out closed, the audit phase running alongside the evaluation harness – is visible in the shape of the chart rather than asserted in the prose.

Table 3.5: Task Timeline, weeks 12 to 48.

Task	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Requirement analysis, corpus characterisation and contract lock																			
Datastore, queue and container infrastructure																			
Ingestion, normalisation and deduplication																			
Stage-1 cheap NLP and sentiment fusion																			

Task	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Router gates and comment selection																			
Stage-2 lanes, prompts and response caching																			
Assembler, canonical schema and three-store fan-out																			
Comment ensemble, abstention and watchlist stance																			
Pluggable backend, roles, privacy lock and retrieval																			
MCP servers, the agent runner and the nine agents																			
React dashboard, live streams and job control																			
Audit passes, regression tests and the evaluation harness																			
Report writing and defence preparation																			

3.5 Summary

This chapter brought together our requirement analysis and design specification. We derived eighteen functional and twelve non-functional requirements from the gaps of Section 2.3, stating each with the design element that satisfies it; the architecture was presented as five bands whose boundaries each trade something for a property the system needs; and the data-flow diagrams fixed the boundary and where state is held, with one deliberate absence at level 0 – nothing is written back to the upstream platform – and one deliberate isolation at level 1, where the corpus-tier insight process is reachable only through persisted stores. Our load-bearing design decision is that the **router makes two independent decisions** rather than one, and separating them is what made the cost behaviour of Chapter 4 measurable at all. Around it sit the eight-labeller ensemble with its abstention protocol, watchlist stance carried inside a prompt already being issued, the pluggable backend with its role system and tenant privacy lock, and the agentic insight layer over eighteen typed tools. We record every significant alternative we weighed in Table 3.3, and the chapter closed with our five-phase plan, its exit criteria and the timeline as we actually worked it.

Implementation and Results

This chapter describes how we implemented the system end to end: the development environment and deployment configuration, the testing and evaluation we carried out across every functional module, and the results we obtained from running the fully deployed stack. We characterise the corpus, present the verification strategy and the full-chain test result, tests the access-control surface separately from the functional one, gives the four measurement studies the project was able to complete honestly – routing and cost, retrieval quality, label provenance and system correctness – walks through the delivered software feature by feature with screenshots of the running system, and closes by discussing what those numbers do and, equally importantly, do not establish.

4.1 Environment Setup

4.1.1 Software environment and dependencies

We chose a modern Python and JavaScript toolchain with locally served inference rather than a metered API, so that the whole stack can be run, measured and reproduced on hardware a student owns. We developed the system on a single workstation running 64-bit Linux with sixteen logical processor cores, a discrete CUDA-capable graphics device and an integrated one, and we took every measurement in this chapter in that environment, under Python 3.12 with Docker for the datastores and Ollama serving the local models.

The system is written in Python 3.12 with a React 19 front end. **FastAPI with Uvicorn** serves the 51-path HTTP API and its generated OpenAPI specification; **React 19 with Vite** builds the dashboard, styled with Tailwind and charting with Chart.js; **Redis Streams** is the durable message bus and simultaneously the cache, deduplication set, counter store and progress channel; **PyTorch and Transformers** run the classifier heads and the sentence encoder; **Ollama** serves four local models, which is what makes the marginal token cost of a run genuinely zero; and **uv** pins the entire dependency tree from one lock file. Supporting tools are Git, Docker, pytest with vitest and Playwright chained into one pass-or-fail command, and oxlint with ruff. The Python 3.12 pin illustrates how a single dependency constrains a project: the optional machine-learning extra cannot be installed on Python 3.13 because a transitive dependency publishes no wheel for it.

Rather than one general-purpose database, we run four stores chosen by access pattern, all in Docker on the development tier: **PostgreSQL 16 with pgvector** for operational rows, job state, the canonical result document and the 768-dimension embedding index in the same row; **ClickHouse** for denormalised analytical events, with a collapsing engine on the per-comment table so a re-analysed comment does not double-count; **Redis 7** for the bus, response cache,

deduplication keys, rate limits, job counters, cancellation flags and usage counters; and **MinIO** for S3-compatible object storage at deterministic keys, so production can substitute a managed service with no code change. We pull four models locally, and they are deliberately **not** the same model – the Stage-1 role runs a four-billion-parameter model because it carries the highest-volume work, Stage 2 and the summary role a seven-billion-parameter model, the agent role a retagged sixteen-thousand-token-context build because tool-calling loops exhaust a four-thousand-token window, and a vision-language model is held for the image path. Beyond those roles, Stage 1 loads six supporting models outside the generative roles – a multilingual sentence encoder, a language identifier, a zero-shot topic and intent classifier, a toxicity head, an emotion head and GLiNER [25] for named-entity recognition, chosen so that entity types are supplied at inference time rather than baked into a checkpoint.

4.1.2 Deployment configuration

The three deployment tiers are configurations of one codebase rather than three systems, and what changes between them is only how many processes a concern is spread across and how scaling is expressed. The **development** tier runs the four datastores in Docker and everything else as host processes – the API, five stage workers, three MCP servers, the agent service and the dashboard, eleven in total – with Ollama serving four local models; running the workers on the host rather than in containers is what makes a code change visible without a rebuild, and is why this tier is the one used for measurement, since the stage functions can be invoked directly with the same code paths the queue would exercise. The **MVP** tier packages the same components as nineteen images against six named volumes on a single network, and the **production** tier expresses them as Kubernetes objects, the only tier in which scaling is declarative rather than manual. The practical consequence is a caveat on every number in this chapter: measurements taken on a single host with local model serving characterise the *logic* of the system – how many calls it makes, which posts it routes, what it retrieves – and not its throughput under production hardware. Where a quantity depends on the deployment shape rather than on the code, Section 4.3.3 records it as unmeasured rather than estimating it.

4.2 Testing and Evaluation / Performance / Comparative Analysis

4.2.1 The corpus

We take every measurement in this chapter over one corpus: the complete post-with-details export that the ingestion path uploads. Table 4.1 characterises it in full, including the parts that are unhelpful to us, because the sampling properties of this corpus are the single largest threat to the validity of anything we build on it.

Table 4.1: Corpus characterisation.

Property	Value
Posts	50, across 25 monitoring campaigns; 37 photo-with-text, 7 photo-only, 6 text-only; 43 carry a caption
Publication window	29 April to 19 May 2026; caption length median 245 characters, maximum 5,427

Property	Value
Comments shipped in the payload	10,272, against 274,126 the platform reports for those posts
Corpus-level comment coverage	3.75 percent; per-post median 26.7 percent, minimum 0.25 percent, 12 posts below 10 percent, 5 above 100 percent
Stored comments per post	minimum 87, median 98, maximum 2,857
Comment reactions	mean 3.97, maximum 946; 64.1 percent of comments have zero reactions
Comment script mix	81.4 percent Bangla script only, 11.1 percent Latin only, 4.6 percent both, 2.9 percent neither; 2.8 percent emoji-only by the pipeline's own kind classifier
Crowd reaction mix	442.6k LIKE, 155.6k SAD, 95.6k LOVE, 67.6k HAHA, 20.2k ANGRY, 10.2k CARE, 2.6k WOW, plus 93,903 shares
Threading	0 comments carry a parent identifier; 1,111 report a non-zero reply count, so replies exist upstream but are not shipped
Image references	69 across 44 posts, all relative object-storage keys whose objects are absent

Three consequences follow, and they are stated here rather than in a limitations section because they constrain every number that comes after.

1. **The comment sample is not random.** The stored comments are the platform's returned set, which is engagement-ordered. With 64.1 percent of comments carrying zero reactions and a maximum of 946, aggregating sentiment over an engagement-biased 3.75 percent sample and calling the result "the sentiment of this thread" is not supportable. What the data supports is a claim about *the most-engaged N comments per post*, and that is the phrasing this report uses.
2. **Post-level sentiment is a text measurement.** The 69 image references are relative object-storage keys whose objects are not present, so no image bytes are reachable in any runnable configuration and the vision term has never contributed a non-zero value. The image path reports an explicit failure status rather than a fabricated neutral verdict, and the fusion weights renormalise over the terms that carry a real model verdict, so an absent image term no longer drags a genuine text signal toward neutral.
3. **We keep the 7 caption-less posts in the corpus.** They yield no post text to analyse, but they carry 1,307 comments that analyse like any other. An earlier working corpus of ours excluded them, which meant we were measuring a population the running system never processes and under-counting the comment lane, now the dominant cost.

Coverage itself deserves one further note, because five posts exceed one hundred percent – up to 112 stored comments against a reported total of 42 – which cannot be replies inflating the numerator, since no comment in the corpus carries a parent identifier. The pipeline clamps coverage to 1.0 and raises an explicit anomaly flag rather than rendering a figure above 100 percent.

4.2.2 Verification strategy and the full-chain result

We automated the testing strategy rather than running it manually, because the failures that dominated this project are not ones a person notices by looking. Our verification has three layers. **Contract tests** assert that documents crossing every boundary validate against their schema, and several grep the source to assert structural invariants – that every worker checks the cancellation flag before doing work, that the dashboard reads only fields the API returns, that the autoscaler manifests name exactly the streams the workers create. Those fail on a rename, which is their purpose. **Regression tests** pin every defect found in the audit passes. **Datastore tests** run against throwaway PostgreSQL and Redis containers started per session, so a default run can never damage a developer's environment; two further suites are gated behind explicit markers.

Those layers hold **1,420 automated tests across 68 files**, plus 96 dashboard unit tests and 2 browser tests. Tables 4.2 and 4.3 give twenty-two *acceptance scenarios*, one or two per module, at a level a reader can check against the screenshots of Section 4.3.1; each is backed by one or more tests, but the suite is far larger, because most of what it asserts is a boundary condition, a malformed input, a component that failed to load, or a rename that must break a build. Its distribution is the clearest statement of where this project's risk sat: the agentic layer carries **325 tests, nearly a quarter of the suite**, for a subsystem that is a small fraction of the code, because it is the only component whose failures are *plausible* – an agent that invents a citation produces an answer that reads exactly like a correct one. Pipeline stages, fusion and routing carry 237; chat and question routing 139; security and data residency 116; the comment ensemble and its provenance 110; retrieval, clustering and reports 109; contracts and the dashboard-to-API surface 147; job control, queues and observability 150; and the audit regressions with watchlist stance 87.

Table 4.2: Functional Test Cases and Results.

Module	Test case	Expected outcome	Result
Authentication	Present an expired token on a streaming endpoint	Rejected on every transport, not only the authorization header	Pass
Ingestion	Upload the same post-with-details payload twice	Validated against the input schema; upsert on the upstream identifier, no duplicate rows	Pass
Stage-1 NLP	Analyse a caption-less post	Comments still analysed; post-level text fields reported absent, not fabricated	Pass
Router	Route a post whose Stage-1 confidence is missing; select comments on an 87-comment thread	Gate fires on absent confidence; 45 duplicates removed, 36 selected, arithmetic on the envelope	Pass
Comment ensemble	Label a comment when a head fails to load, and one no model read	Head abstains and the voter count falls; the unread comment reports uncertain at zero voters	Pass
Watchlist stance	Score a comment naming two entities in opposite clauses	Each entity scored from its own clause, not from a character window	Pass
Assembler	Assemble a result that violates the output schema	Assembler raises; nothing is persisted to any store	Pass

Module	Test case	Expected outcome	Result
Job control	Stop, then resume, a running analysis job	Cancellation bounded at one post per stage; 29 posts analysed, 21 re-enqueued, none repeated	Pass
Search	Query with an exact post identifier in semantic mode	Identifier lookup short-circuits ahead of the requested mode	Pass
Agent layer	Repeat a tool call byte-identically; cite a post no tool returned	Second call refused and untried tools named; the citation is listed as unverified beside the answer	Pass
Dashboard and telemetry	Subscribe late to a job's event stream; issue an LLM call on either backend	Buffered events replayed and duplicates dropped by sequence number; backend, role, model, tokens and latency recorded in the ledger	Pass

The whole chain runs as a single command in which each stage must pass for the next to start, so a completed run is one pass-or-fail signal rather than five. On 29 August 2026 it passed in seventy-three seconds: 1,417 Python tests with three skips, 96 dashboard unit tests, 2 browser tests, lint on both languages and a production build. Test code is now 19,850 of the repository's 56,789 Python lines – 35 percent, up from 14 before the audit passes – and almost all the growth is regression tests pinning audit findings. We ran a mutation pass on ten repaired defects and confirmed that nine were caught when reverted; the tenth, the watchlist alias deduplication, was not, so that fix had no coverage at all, and we have now tested it. The value of the regression layer is in what the tests are named: one module holds 38 of the 49 audit regressions, each name stating the defect it pins, so the file reads as a list of the mistakes this system has already made once.

4.2.3 Security and access-control testing

Functional correctness establishes that each module does what it is meant to do; it does not establish that a caller who is not entitled to a result cannot obtain one. The two are tested separately because they fail differently: a functional defect produces a wrong answer, whereas an access-control defect produces a *correct* answer delivered to the wrong party. Every identity-bearing path – the token issuer and verifier, the API-key store, the single-use streaming ticket, the tenant scope on every analytical query, and the data-residency policy deciding which inference backend a tenant's text may reach – was exercised against both the permitted and the refused case, on the principle that a check never observed to refuse anything has not been tested at all. Eight suites carry that work inside the same pass-or-fail chain, collecting **138 tests, all passing in 6.89 seconds**: seventeen on token issue and verification, thirty-seven on signup and passwords, twenty-five on transport hardening, seventeen on the backend policy resolver, fourteen asserting the privacy lock survives every pipeline entry point, two on cross-tenant reads, four on rate limiting and twenty-two on prompt-injection containment.

Table 4.3: Security and Access Control Test Results.

Test scenario	Surface tested	Expected behaviour	Result
Present an expired token, then a forged one	Token verification	Both refused with 401; a forged signature never reaches the handler	Pass
Claim a privileged role inside a client-supplied token body	Principal construction	The authentication-method claim cannot be set by the client; claims are allowlisted	Pass
Authenticate with an API key	Hashed key store	Key looked up by SHA-256 hash and never by value; the tenant is taken from the stored row	Pass
Open a stream with a ticket, then replay it; open one malformed	Streaming authentication	First open succeeds, the replay is refused, and a malformed ticket gives 401 rather than a silent unauthenticated stream	Pass
Register an account, then inspect what was stored	Signup path	Password never stored in plain text; extra tenant and role fields in the body do not become attributes	Pass
Read analysis results as tenant B that were written by tenant A	Tenant isolation	No rows returned; every analysis-result query is tenant-scoped	Pass
Request the metered cloud backend as a privacy-locked tenant	Data-residency policy	Refused rather than silently downgraded, on both job entry paths, with the backend stamped into the envelope	Pass
Request the metered backend while the policy table is unreadable	Data-residency policy	Denied: the check fails closed rather than defaulting permissively	Pass
Exceed the configured request rate on one identity	Rate limiting	Blocked at the limit, released on window rollover, unaffected by other identities	Pass
Embed an instruction in retrieved comment text; ask the agent for its system message	Prompt-injection containment	Wrapped as untrusted data with forged delimiters neutralised; the system message is refused by policy	Pass

Two rows are worth reading twice, because they are where a system usually fails quietly rather than loudly. The first is the unreadable policy table: a residency check that cannot reach its policy store has two options, and the tempting one – carry on with the previous answer – converts a storage hiccup into a data-egress incident that leaves no trace. The test named `test_an_unreadable_policy_table_denies_rather_than_permits` pins the other choice. The second is the explicit backend request from a locked tenant, which is **refused rather than downgraded**, because silently substituting the self-hosted backend would honour the policy while lying to the caller about which model answered. What this subsection does not claim is a penetration test: these are white-box tests written by the author of the code they guard, so they establish that the checks behave as specified under the conditions the author imagined, and an adversarial review by a party with no knowledge of the implementation is named in Section 6.3 as the work that would close the remaining distance.

4.2.4 Routing behaviour and the cost split

We ran the routing measurement over the **real stage functions** – normalisation, text and image analysis, fusion, result construction and the gate itself – with no options set, reporting the router's own counters. We ran it twice with the same code and two different Stage-1 engines, because the comparison between them is the result. The quantity that decides how much the router is worth is not the routing rate but the **share of large-model spend the gate governs**, which is the post-level lane's share of all calls.

$$\gamma = C_{\text{gated}} / (C_{\text{gated}} + C_{\text{comment}}) \quad (4.1)$$

where C_{gated} counts the large-model calls the router's gate releases – the post-level lane together with the post-level Stage-2 tasks – and C_{comment} counts the per-comment labelling calls, which run for every post whether the gate routed it or not. The measured value is $\gamma = 0.304$ under the shipped configuration and $\gamma = 0.547$ under the keyword baseline.

Table 4.4: Measured routing and large-model call split, 43-post captioned corpus, cold cache, 25 comments per batch.

Quantity	Keyword-stub Stage 1	LLM Stage 1 (as shipped)
Posts routed for post-level work	32 of 43 (74 percent)	7 of 43 (16 percent)
Post-level calls (summary, insight, post type, comment summary)	124 (15.3 percent)	22 (4.2 percent)
Comment-level calls	689 (84.7 percent)	507 (95.8 percent)
- Stage-1 comment labelling, all 43 posts, which the gate cannot reduce	368	368
- Stage-2 comment stance, routed posts only	321	139
Total calls per corpus run	813	529
Share of all calls the routing gate governs	54.7 percent	30.4 percent

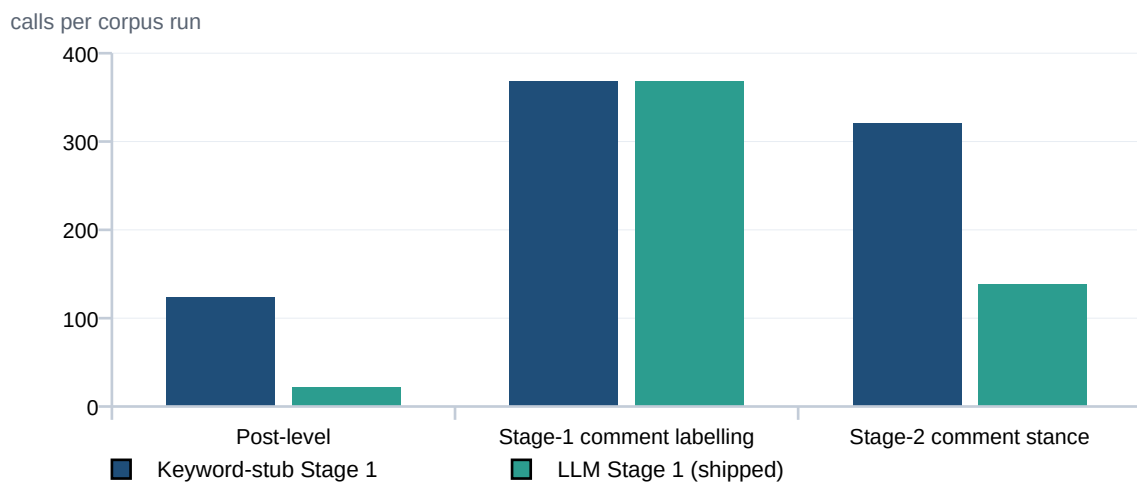


Figure 4.1: Where the large-model calls actually go.

Reading the chart. The middle pair of bars is the finding. Stage-1 comment labelling runs for every post whether the gate routed it or not, so it is identical in both configurations and the routing gate cannot reduce it at all. Improving Stage 1 halves the post-level bar and cuts the Stage-2 stance bar by more than half, but leaves the largest single bar untouched, which is why the gate's share of total spend falls from 54.7 to 30.4 percent while the absolute bill falls only from 813 calls to 529. What sets the bill is how many comments exist.

Three conclusions follow, and the first contradicted our own headline claim. **The routing rate is not the cost story:** between 85 and 96 percent of calls are comment-level. The initial claim was that cheap models perform ninety to ninety-five percent of the work and a selective large model handles the rest, and that this was the central cost-control idea; that assertion was never measured, and when it was, the gate had in fact been routing one hundred percent of posts for a period because of two field-name defects – and after those were repaired it still governed only a minority of spend. The claim that survives is narrower and is the one this report makes: **cheap NLP filters which comments and which posts deserve a large model**, and the second half is now the bigger one. Second, **improving Stage 1 shrinks the gate's importance rather than the bill**, which runs counter to intuition: a better Stage 1 types more posts confidently, so fewer route and the gate decides less, while comment labelling runs unchanged. Third, **the gate selects the right posts** – under the shipped configuration the 7 routed posts hold 3,404 of the 8,713 comments that reach a model, so the gate is escalating the large, contentious threads, which is the behaviour the design intends and which a routing rate quoted on its own cannot show.

Measurement also corrected one of our earlier estimates. We had expected filtering emoji-only comments out of model batches to save roughly seventeen percent of the comment bill; emoji-only comments are **2.8 percent** of the corpus. The seventeen-percent figure was the share taking the fast path, which conflates emoji-only reactions with short text comments, and a short text comment still carries text a model can read. Filtering emoji remains correct because they are pure waste, but it is not a cost lever.

4.2.5 Retrieval evaluation

Retrieval is the one axis we evaluate against a ground-truth set. We derived a 32-query known-item set from each target post's own topics and entities, with exactly one marked relevant post per query, and ran the same set before and after enabling real embeddings, so the two recall columns of Table 4.5 differ only in whether the stored vectors are semantic or hashes. Three standard measures are reported over the query set at cut-off $k = 10$. Because each query has exactly one relevant document, $\text{recall}@k$ reduces to the fraction of queries whose target appears in the top k at all, as in Equation (4.2); the two rank-sensitive measures are the mean reciprocal rank, $\text{MRR}@k = (1/N) \sum 1/r_q$, and the normalised discounted cumulative gain, $\text{nDCG}@k = (1/N) \sum 1/\log_2(1 + r_q)$, both of which reward the target appearing higher in the list rather than merely appearing.

$$\text{recall}_k = (1 / N) \cdot \sum 1[r_q \leq k] \quad (4.2)$$

The sum runs over the $N = 32$ queries q . r_q is the position of the one relevant post in the list returned for query q , and $1[\cdot]$ is the indicator function, worth 1 when the condition holds and 0 otherwise. This is the measure that matters when the whole top- k is handed to a generator [42], because a target absent from the context cannot be recovered by the model.

Table 4.5: Retrieval evaluation over 32 known-item queries, $k = 10$, on the 50-post corpus.

Configuration	recall@10 (hash vectors)	recall@10 (real)	MRR@10 (real)	nDCG@10 (real)
vector (dense only)	0.1875	0.6250	0.2869	0.3667
lexical (full text fused with trigram)	0.3750	0.7500	0.6503	0.6748
chunk (dense over passage chunks)	not measured	0.6562	0.3014	0.3846
hybrid (dense fused with lexical)	0.5312	0.8750	0.6108	0.6759
chunked (chunk fused with full text and trigram)	not measured	0.8750	0.6482	0.7041

Three findings come out of that table, and the third is a defect rather than a result. **We found hash vectors indistinguishable from chance:** real embeddings are worth 3.3 times on the dense arm alone, from 0.1875 to 0.6250, and the end-to-end configuration moved from 0.5312 to 0.8750, a 65 percent relative gain. This matters beyond retrieval, because the same vectors drive the clustering the report layer summarises, and clustering hash vectors groups posts at random and then pays a large model to describe the grouping. **Chunking improves ranking rather than recall on this corpus,** because the corpus consists predominantly of short captions. And **the lexical arm's improvement turned out to be a bug fix, not a benefit:** the lexical column moved from 0.3750 to 0.7500 with no vector involved, because the full-text half of that arm had never fired at all – the query builder conjoined every term, so an eight-token question demanded all eight words in one caption and matched **zero** rows across the entire query set. Everything we had credited to "full text plus trigram" was trigram alone. We record it because it is the same failure shape as the routing defect: a component that produced no output, reported no error, and was covered by a passing test.

A stratified view of the same 32 queries changes how the headline table should be read, and does so in the embeddings' favour. Splitting the queries by the lexical overlap between query and target gives two populations. On the twelve near-verbatim lookups, lexical retrieval cannot fail and scores 1.000, which is why it outscores hybrid on MRR overall. On the eight genuinely leak-free queries – where the query shares no vocabulary with its target – **lexical scores exactly zero** and the dense arm is the only retriever that returns the target at all; the fused configuration ties lexical on the easy bucket and beats it 0.800 to 0.600 on the hard one. Neither arm dominates, which is precisely the premise the fusion design rests on, and it is invisible in the aggregate.

4.2.6 Label provenance and ensemble coverage

A sentiment chart that cannot say what produced its numbers is not evidence, so we record provenance per comment and per post. Measured over all 10,272 comments in the default configuration, we found that **28.7 percent of per-comment labels are model output and 71.3 percent are not**: partly a fourteen-word emoji-and-lexicon rule, and mostly a deterministic function deriving a label from a hash of the first fifty characters. That function is reproducible and it is not sentiment. Before we recorded provenance, the same run reported 8,513 model inferences in a configuration where zero models had been loaded, because the classifier tagged every substantive comment as model output regardless of whether a model existed. Our remedy was not to hide the slice but to label it.

Four reporting mechanisms follow from that measurement, and we made them permanent parts of the output contract. Each comment carries a **method** naming the engine that actually ran and a **kind** distinguishing emoji, short and substantive text. Each post carries a **provenance block** stating what share of its labels were model inferences. Each analysed comment carries the **number of voters that actually spoke** alongside its label and agreement figure. Finally, a comment that no model read is reported as **uncertain at zero voters**, never as neutral.

4.2.7 Comparative analysis

Set against its two natural baselines on the quantities we measured, our shipped configuration reads as follows. The all-cheap keyword baseline routes 32 of 43 posts (74 percent) for 813 calls, of which the gate governs 54.7 percent; the shipped hybrid routes 7 of 43 (16 percent) for 529 calls, of which the gate governs 30.4 percent; and an LLM-everywhere baseline routes all 43 by definition. The comparison is deliberately restricted to measured quantities – an accuracy column is absent because labelling accuracy sits outside the evaluation scope.

Four readings follow, and the first costs us our simplest claim. Cutting total calls from 813 to 529 is a reduction of roughly a third – real, but nowhere near the order-of-magnitude saving that a routing rate falling from 74 to 16 percent appears to promise, because as the gate routes fewer posts it also governs less of what remains. Second, we leave the LLM-everywhere column as a bound rather than an estimate: we did not run that baseline, and a column of plausible numbers beside two of measured ones would have invited exactly the comparison the evidence does not support. Third, two properties favour the shipped configuration on grounds that have nothing to do with cost – independent verdicts per comment rise from one to as many as eight with abstention, and the output can state what produced each label – and both belong to the ensemble rather than the router, so the contribution does not rest on the cost claim that measurement weakened. Fourth, retrieval recall is orthogonal: it reads persisted analysis rather than participating in the cascade, and its improvement from 0.5312 to 0.8750 came from replacing hash vectors with real embeddings, not from anything the router does.

4.3 Results and Discussion

This section reports what was delivered, what the measurements of Section 4.2 establish, what they disconfirm, and where the evidence stops. Section 4.3.1 walks the delivered software feature by feature against the objectives of Section 1.3 and the requirements of Section 3.1.3; Section 4.3.2 draws the findings; Section 4.3.3 states the boundary of the evaluation and its threats to validity.

4.3.1 The delivered system, in the running software

Every figure in this section is a screenshot of our running system, captured on 29 August 2026 with the corpus of Section 4.2.1 loaded. Nothing is a mock-up: where a panel shows a number, the pipeline of Chapter 3 computed it. We start the whole stack from one command that brings up the four datastores, verifies that the sentence encoder produces real vectors rather than hash stubs, and launches eleven host processes over a self-describing API of 51 documented paths.

The Overview tab of Figure 4.2 is the first surface an operator sees, and its token-economics panel is the half that matters to this project's central argument: five lanes divide 941 calls as post 10, comment 284, Stage-2 batch 68, agent 39 and interactive 540, with the cost incurred stated beside the cost the same work would have metered. Two design decisions sit in the stat cards themselves – the routing rate is printed beneath the count of posts routed rather than left to be derived, and corpus coverage is a first-class card rather than a footnote.

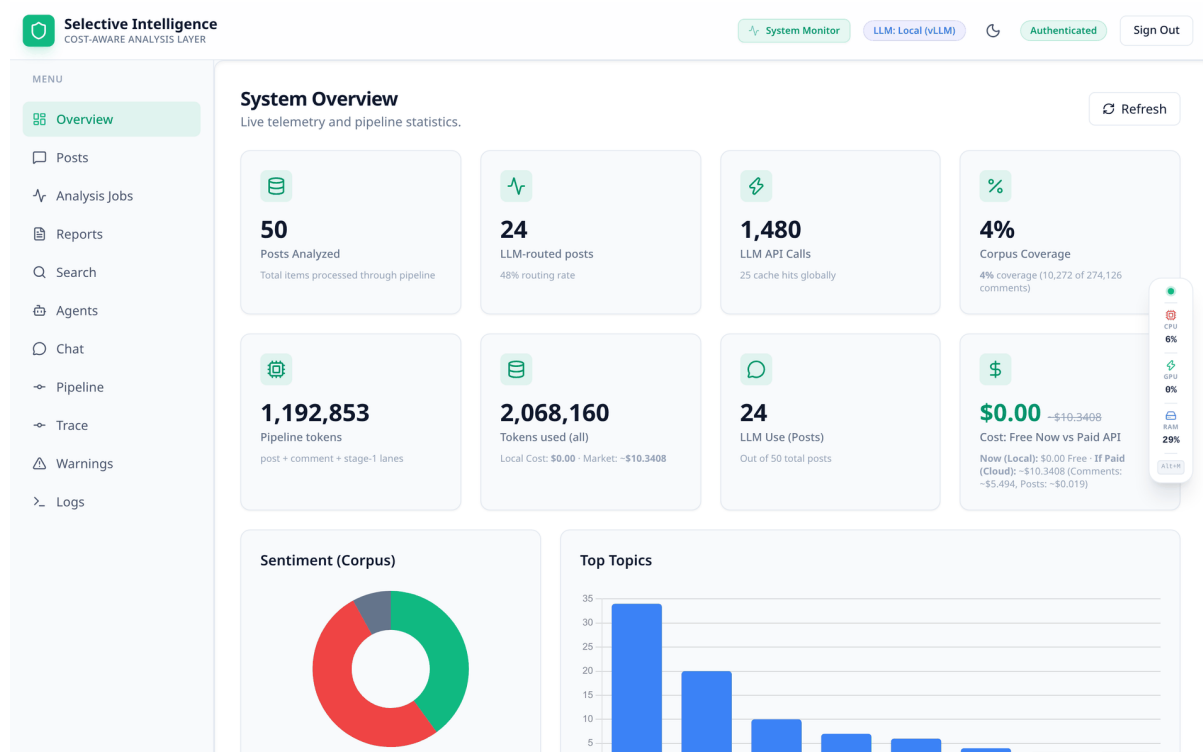


Figure 4.2: Dashboard, Overview tab: the corpus-level summary and the token-economics panel.

In Figure 4.2, the Overview tab is shown in three bands: eight stat cards, four charts (corpus sentiment, top topics, comment emotions and language distribution), and the token-economics panel that breaks spend down by lane. Every figure on the tab is derived at read time from D2 and D5, written by Process 5.0 (Assemble) of Figure 3.3 on flows [12] and [13], rather than from any maintained counter – so a tab that disagrees with the database cannot happen. The coverage card states 10,272 comments analysed of 274,126

reported, keeping the sampling ratio of Section 4.2.1 on the first screen rather than in a footnote.

Figure 4.3 is the explorer over that corpus. Its comment column shows analysed, stored and platform-reported counts together, so the sampling ratio is visible in the table itself, and an exact identifier typed into the search box bypasses search and opens the post directly. Opening a row gives the post detail modal, where the system's verdict sits beside the upstream platform's coarse score under an amber warning that only 87 of the platform's 270 reported comments were ever scraped.

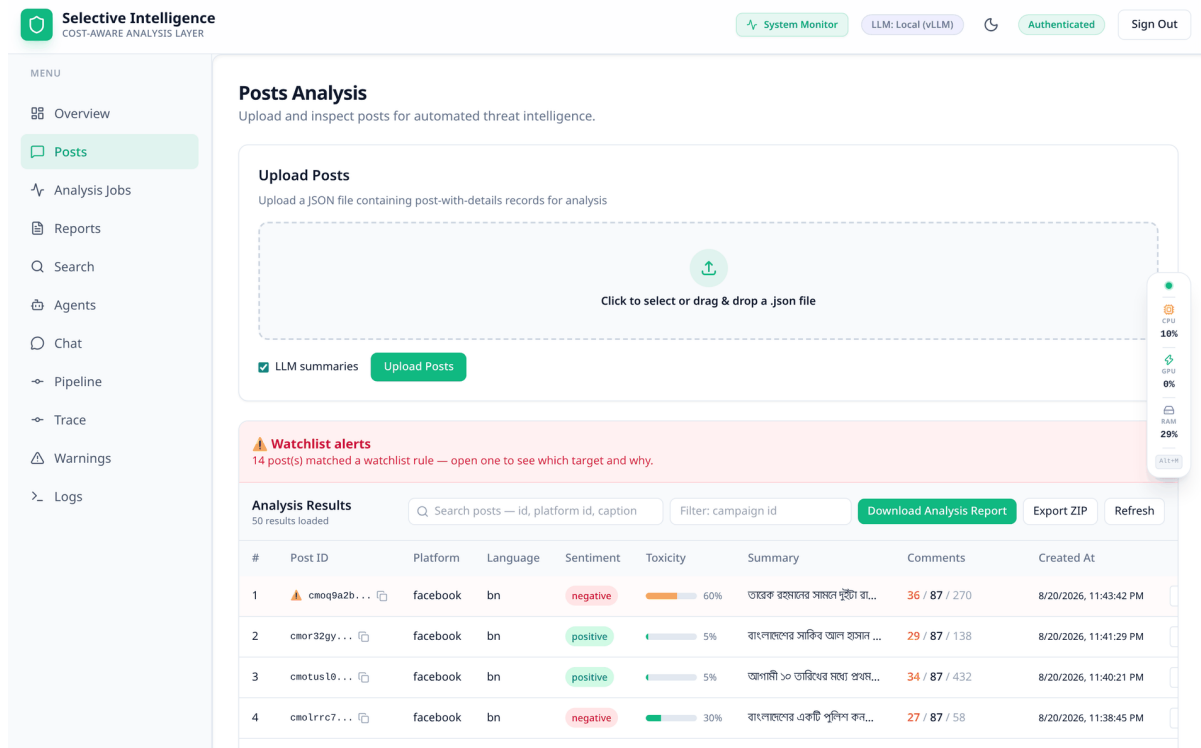


Figure 4.3: The Posts tab: upload, watchlist banner, and the analysed corpus with its coverage arithmetic.

In Figure 4.3, the Posts tab is shown in three regions: the upload zone that takes a post-with-details JSON file, the watchlist banner counting the posts that matched a target, and the analysed corpus as a table whose comment column carries analysed, stored and platform-reported counts side by side. The rows are read from D2 (analysis_results and embeddings) of Figure 3.3, written by Process 5.0 (Assemble) on flow [12]. That three-part column is the coverage arithmetic of Section 4.2.1 made visible in the table itself, so the sampling cannot be missed by a reader who never opens a post.

Scrolling that modal reaches the comment thread, and Figure 4.4 is what the eight-labeller ensemble of Section 3.2.3 looks like when it is disclosed rather than fused away: each comment carries its eight verdicts side by side, the combined label, the score and the voter count. This is the surface that makes abstention legible, because a comment no model read reports as *uncertain* at zero voters, visibly different from one seven labellers read and agreed was neutral – a difference a single fused label cannot express.

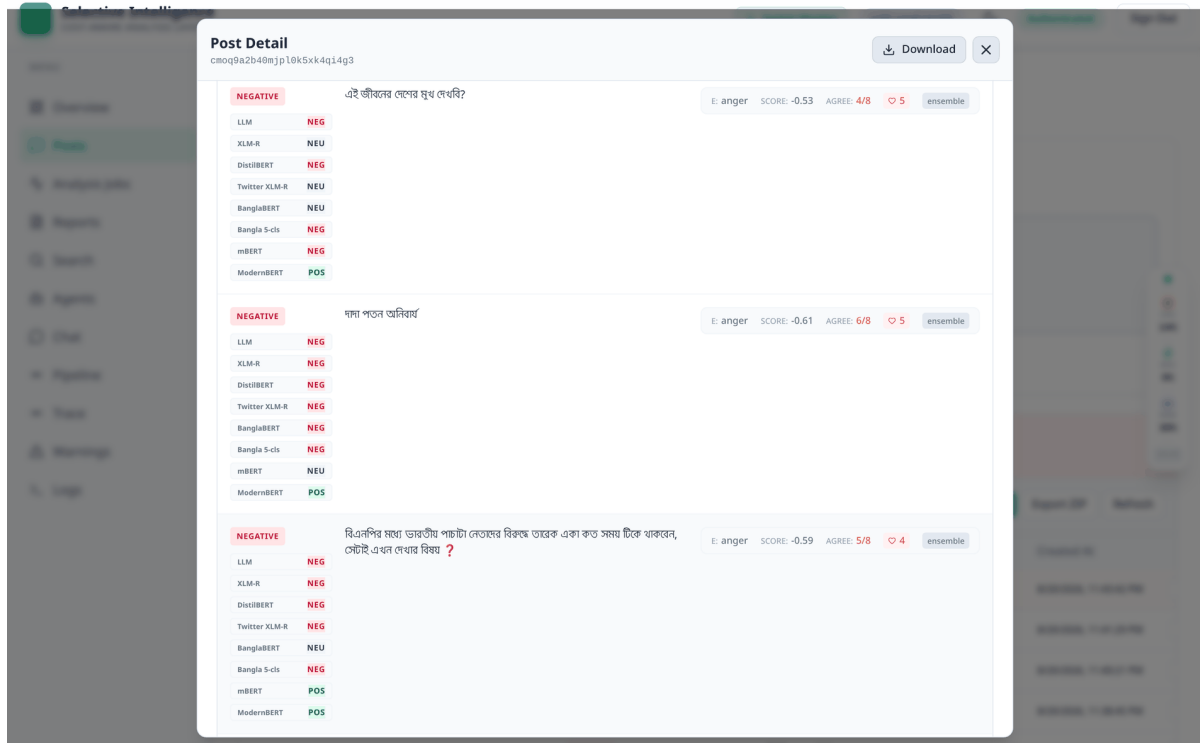


Figure 4.4: Three comments, twenty-four votes: what per-labeller disclosure looks like.

In Figure 4.4, three consecutive comment rows carry twenty-four votes between them: each gives the combined label, the eight individual verdicts naming labeller and vote, the comment text, and a metric strip of dominant emotion, combined score, agreement fraction, reactions and the source tag. These labels come from Process 4.0 of Figure 3.3 and are read back from D5, written by Process 5.0 on flow [13]. The second row is the interesting case: two heads vote positive against five negative and one neutral, and the combined label reports 5 of 8 rather than presenting itself as certain.

Re-analysis is triggered from the Analysis Jobs tab or the API, both producing the same job record with track, stop, resume, re-run and delete; resume exists because a stopped job leaves posts already analysed and re-enqueuing them would repeat their cost. The Pipeline tab reads in-flight, backlog and dead-letter counts from the Redis consumer groups themselves rather than estimating them, and the Logs tab tails one capped buffer into which all eleven processes write.

The Trace tab of Figure 4.5 re-runs one stored post through the real pipeline and follows it live, and it is the surface that makes the router's decision inspectable rather than inferable: five layers, each holding the payload that stage actually emitted, with the gate's own inputs printed beside its verdict and explicit *skipped* frames where the post-level lane was bypassed. A gate that reports which of its six rules fired, on the same screen as the inputs those rules read, is a gate whose behaviour can be disputed – which is precisely how we found the routing defect of Section 4.3.2.

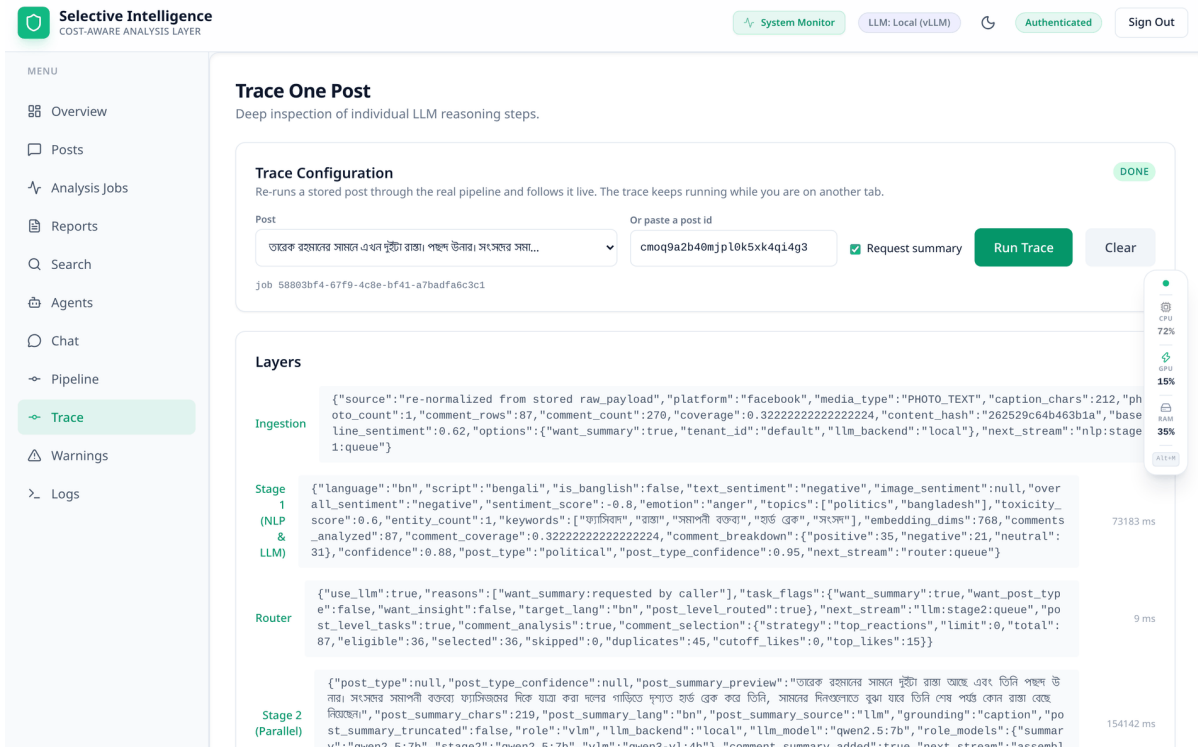


Figure 4.5: Trace tab: one post through all five stages, with each stage's own payload.

In Figure 4.5, one post passes through all five stages, each layer holding the payload that stage emitted – ingestion's source and comment counts, Stage 1's language, sentiment, topics, toxicity and embedding, the router's use_llm with the reasons that fired and the selection arithmetic, Stage 2's models per role, and the assembler's schema version and fan-out. The five layers are Processes 1.0 to 5.0 of Figure 3.3 executing in order on flows [4], [6], [7] and [10], ending in writes to D1, D2 and D5 on flows [11] to [13]. The run shown selected 36 of 87 comments after removing 45 duplicates.

The Search tab exposes the two retrieval paths of Section 3.2.7 side by side, and what the semantic view adds is provenance on the retrieval itself: the mode line names the active encoder at 768 dimensions, and an amber banner warns when results carry hash-based stub vectors – the interface expression of Section 4.2.5, since a result set drawn from stub vectors is not a worse ranking but a meaningless one. Above it the agent layer discovers its eighteen tools rather than declaring them, and its execution trace shows one entry per tool call with the server that served it; a *skipped* chip marks the byte-identical repeat calls the runner refuses.

Figure 4.6 is the most useful capture in this chapter, because in it the model got the answer wrong and the system said so. The agent called `reaction_mix`, received the real distribution, wrote a correct table from it, then attached post identifiers appearing in no tool result; the verification block lists them as unverified citations. It does not stop a small model fabricating – nothing at the prompt level can – but it makes the fabrication visible in the same view as the answer, which is all we claim for it.

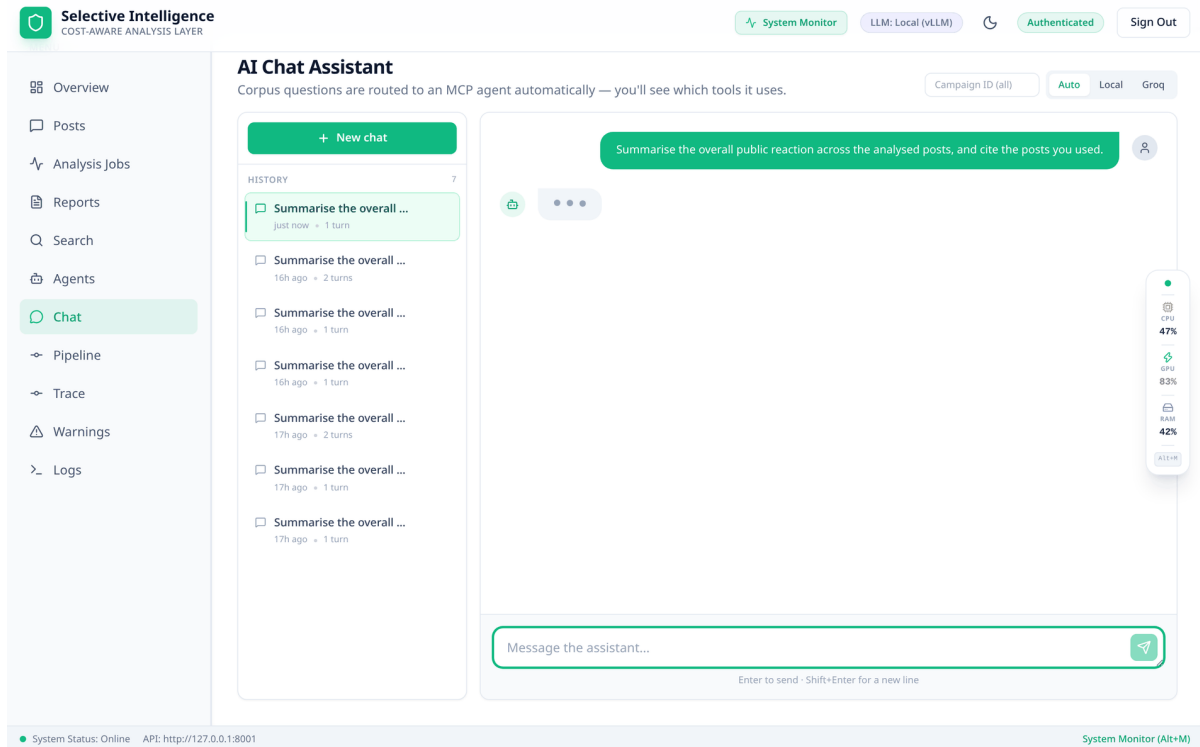


Figure 4.6: The Chat tab catching its own model inventing citations.

In Figure 4.6, the foot of a completed chat answer shows the reaction table the agent wrote from the tool result, then two verification blocks – unverified citations, listing post identifiers no tool returned, and unverified quotes. The answer came from Process 6.0 of Figure 3.3, and verification compares every citation against what flows [15] and [16] actually returned during the run. The percentages are real, having come from reaction_mix; the citations are not. Verification is per claim rather than per answer, so a reply can be right in its statistics and wrong in its citations, and be displayed as exactly that.

Two surfaces close the walkthrough. The watchlist alert table states which target matched and under which rule, which is what makes an alert falsifiable, and the System Monitor drawer streams per-core load, both graphics devices, memory and thermals, so a slow run can be attributed to saturation rather than a stalled stage. The cost ledger returned by `/v1/usage` – corpus counters, then tokens and cost by backend and model, then the five-lane split – is where the argument of this report is finally settled or lost, and it is the artefact that made the finding of Section 4.2.4 visible to us: the lane rows, not the routing rate, are where the money is. Table 4.6 then indexes every delivered capability against where it lives and the class of evidence supporting it.

4.3.2 What the measurements establish, and what they disconfirm

Our measurements support three findings. The cascade works as a mechanism, and we can now observe its behaviour rather than assert it: the gate responds to Stage-1 quality in the direction the design predicts, its decision travels explicitly on the envelope, and its bypass branch is validated end to end against the output schema – a branch that had never once executed before the audit that found it. Hybrid retrieval is a real improvement, and the stratification of Section 4.2.5 explains why: the fused configuration reaches 0.8750 recall@10, and the two arms fail on *disjoint* query populations. Provenance reporting changed what we can honestly claim, and asymmetrically – it removed claims rather than adding them, turning two apparent successes into stated absences: the image modality, and the majority of the default configuration's comment labels.

Our original headline claim was wrong twice over, and reporting that is more useful than reporting only the corrected version. It was wrong **in fact**, because the gate was routing every post: two rules read field names Stage 1 never emitted, one read a field Stage 1 sets as a placeholder for Stage 2 to fill, and one compared a language value against a code-mixing marker recorded under a different field – net, one rule always fired and three could never fire. Every one of the four was a **silent degradation rather than an exception**, which is the generalisable lesson we take from this project; we then applied the mitigation – named reader functions, a diagnostic log routed through those same readers, and a test asserting that the reader sees the producer's real output – to the three other places the identical pattern had occurred. It was wrong **in framing**, because even a correctly functioning gate governs a minority of spend once every comment is labelled. That is the finding with the most transfer value: **when the routed unit contains many sub-items that each need work, routing the container is not the cost lever**. The published cascade literature routes single queries [30], [31], [32] and does not encounter this, which is why it took a measurement rather than a reading for us to find it.

4.3.3 The boundary of the evaluation, and threats to validity

Labelling accuracy is outside the scope of this project, and we claim none. What we deliver is a system: one that routes work, labels it, records what produced every label, and serves the result under a validated contract. A benchmark number would require human-adjudicated labels, and producing them is an annotation programme with its own guideline, agreement study and timeline rather than a further measurement script. The tooling ships with the repository, but the gold file holds **zero adjudicated labels**, deliberately, because seeding them from a model in this repository and then scoring that model would measure agreement with itself. A green suite is not a quality claim, so the report quantifies rates, counts, coverage, latency, retrieval quality and provenance, and quotes no macro-F1 for any labeller. Four further capabilities cannot currently produce a signal, and we list rather than omit them: the image modality, because no image bytes are reachable; horizontal scaling, for which we have timed no multi-replica run; cross-backend quality parity, under the same annotation dependency; and the cross-encoder reranker, for which the harness prints a skip.

Table 4.6 is our evidence ledger for the system as a whole: every capability we delivered appears exactly once, with the module it lives in and the class of evidence supporting it, so no claim in

this report rests on a stronger basis than the ledger records. Four classes are used – *measured*, *works but unmeasured*, *unexercised*, and *out of scope*.

Table 4.6: Delivered capabilities, where each lives, and the class of evidence supporting it.

Capability	Where it lives	Evidence class	Basis
One-command start; self-describing API of 51 paths / 63 operations	<code>run_all.py</code> ; <code>services/api</code>	Measured	Full-chain run, Section 4.2.2
Routing rate and rules fired per Stage-1 engine; post-level versus comment-level call split	<code>services/router</code> ; five-lane counters	Measured	Router counters over the real stage functions, Section 4.2.4
Per-label provenance and inferred share; comment kinds	Provenance block; Stage-1 kind classifier	Measured	Section 4.2.6; Table 4.1
Coverage arithmetic, clamping and anomaly flags	<code>assembler</code> , <code>Posts.jsx</code>	Measured	Corpus-level and per-post computation, Figure 4.3
Retrieval recall@k, MRR@k and nDCG@k	<code>pgvector</code> , <code>retrieval_mcp</code>	Measured	32 known-item queries, Section 4.2.5
Schema-validated canonical result document	<code>assembler</code> , JSON Schema 1.3	Measured	Validates on every post
Job control: track, stop, resume, re-run, delete; queue depth and dead letters	<code>routers/analysis.py</code> , <code>/v1/pipeline/stats</code>	Measured	29 posts analysed, 21 re-enqueued, none repeated; counts read from the consumer groups
Per-post live execution trace	<code>libs/progress.py</code> , <code>Trace.jsx</code>	Measured	Figure 4.5, with a stated failure mode past the five-minute stream ceiling
Stream and consumer-group identifier agreement; truncation recovery	<code>libs/streams.py</code> ; Stage-2 combiner	Measured	Manifest-versus-worker test; verified firing on a real Bangla post
Ticketed event streams and access control	<code>/v1/auth/sse-ticket</code> plus five endpoints	Measured	138 access-control tests, Section 4.2.3
Eight-labeller comment ensemble with abstention	<code>stage2_llm</code> , <code>PostModal.jsx</code>	Works, unmeasured	Scoring needs adjudicated labels. Figure 4.4
Watchlist target stance, alias matching	<code>libs/stance</code> , <code>Warnings.jsx</code>	Works, unmeasured	36 matcher tests; the validation set of Section 6.3 is not done
Nine agents over eighteen typed MCP tools; chat with citation verification	<code>services/agents/runner.py</code> , <code>routers/chat.py</code>	Works, unmeasured	Hardening is tested; groundedness is not scored. Figure 4.6
Report generation, log aggregation, host telemetry, near-duplicate reuse	<code>routers/reports.py</code> , <code>libs/logging</code> , Stage-2 cache	Works, unmeasured	All run; report value depends on the vectors being real, and duplicate reuse is off by default
Image sentiment, OCR and vision-grounded summary	<code>stage1_nlp/vision_analyzer.py</code>	Unexercised	No image bytes are reachable in any runnable configuration

Capability	Where it lives	Evidence class	Basis
KEDA autoscaling, horizontal throughput, cross-encoder reranking, Grafana dashboards	deploy/k8s/, retrieval_mcp, deploy/	Unexercised	Manifests are test-asserted but untimed; the reranker harness prints a skip; no dashboards provisioned
Labelling accuracy, backend quality parity, gate calibration	eval/ sampler and scorer	Out of scope	Annotation programme specified in Section 6.3

Construct: our corpus is an engagement-ordered 3.75 percent sample, so thread-level aggregates describe the most-engaged comments rather than the thread. *Internal:* we took several measurements with a component degraded, and we name the degradation on each. *External:* fifty posts on one platform supports a claim about mechanism, not population. *Conclusion:* our retrieval figures rest on 32 machine-derived queries.

4.4 Summary

This chapter described how we implemented and evaluated the system, which verifies as a single pass-or-fail chain in seventy-three seconds. The routing gate works and we can observe its behaviour, but it governs 30 percent of large-model calls rather than the great majority – a finding that contradicted our own headline claim. Hybrid retrieval reaches recall@10 of 0.8750 against 0.1875 for hash-seeded vectors, and provenance reporting established that most of the default configuration's comment labels are inferred rather than read.

Engineering Standards and Design Challenges

This chapter records the standards the system complies with and the alternatives we rejected in each case, assesses our work's impact on life, society, the environment and its own sustainability, presents a financial analysis with a costed alternative, and maps our work onto the complex-engineering-problem, knowledge-profile and complex-engineering-activity frameworks.

5.1 Compliance with the Standards

We list only the standards that genuinely bear on this system, and for each we name the alternative we considered with its trade-off, because a standard adopted without an alternative in mind is a default rather than a decision.

5.1.1 Software Standards

For **data interchange** we use JSON validated against a published JSON Schema, and the assembler **raises** on a failure rather than persisting the document, so nothing downstream ever sees an unvalidated result; one increment in our schema version history exists purely because a language-model task's output was computed, paid for and then dropped by a component reading a different set of fields. *Alternatives:* **Protocol Buffers** or **Avro** would give a smaller wire format and stronger typing, but the upstream platform emits JSON, our consumers are third-party applications for which a self-describing text format is far easier to adopt, and the payloads are dominated by free text. The cost is verbosity, and we pay it down where it hurts: our stream payloads escape no non-ASCII characters, which on Bengali text we measured as roughly half the token bill on the agent path.

The **web API** is REST over HTTPS, versioned, and described by a generated OpenAPI document served by the application rather than maintained by hand: endpoints live under a version prefix, asynchronous work returns an accepted status with a job identifier, writes accept an idempotency key, and errors use one envelope. *Alternatives:* **GraphQL** would let the dashboard fetch exactly the fields it needs and would have avoided one defect class hit repeatedly here, where a hand-maintained response model silently dropped computed fields; we rejected it because our consumers are heterogeneous third-party services, caching and rate-limiting are simpler on REST, and a typed response model is the correct fix for that defect class – which is what we did. **Streaming** uses Server-Sent Events across five surfaces; **WebSockets** are bidirectional and would have been the obvious choice, but every surface here is server-to-client only, and Server-Sent Events reconnect automatically over ordinary HTTP. Their one real

disadvantage drove a design decision: an event-source connection cannot carry an authorization header, so rather than putting a credential in the URL we issue a **single-use ticket valid for about sixty seconds**.

Authentication uses JSON Web Tokens with HMAC signatures alongside hashed API keys, with short expiry and a refresh endpoint, and the tenant identity read from the stored row rather than from anything the client sends. *Alternatives:* **OAuth 2.0 with an external identity provider** is the right answer for a multi-organisation deployment and is the documented upgrade path, but adds a dependency our offline test suite cannot exercise; we rejected **session cookies** because our primary consumers are machine clients. The compliance point is the enforcement rather than the scheme: a credential *shaped* like a token is verified as a token on every transport, because verifying it on only one meant an expired or forged token authenticated on all four streaming endpoints. **Character encoding** is Unicode under NFC normalisation applied at ingestion with script tagging – not administrative, since without it two visually identical Bangla strings can differ byte-for-byte, defeating both content-hash deduplication and the alias matcher.

5.1.2 Hardware Standards

On **accelerators and precision**, inference targets CUDA-capable devices with quantised weights, and the deployment is designed so the cheap classifier heads run on **CPU** while the accelerator serves the language model. That is a decision rather than a fallback: we measured that on a four-gigabyte card already serving a seven-billion-parameter model roughly 285 megabytes remain free, which fits one classifier head and not seven, and placing all seven produces the least desirable outcome – some load, some fail, and a head that fails to load abstains silently for the life of the process, so a run silently degrades from eight voters to two. Our initial configuration placed every model on the accelerator and produced exactly that; we rejected running the language model on CPU as well on latency grounds. The split we chose costs roughly three gigabytes of checkpoint storage and a few CPU cores, and makes the ensemble's coverage deterministic.

On **storage and memory**, the vector column is fixed-dimension and the embedding dimension must match it, enforced rather than documented: a mismatch fails every insert instead of silently storing an unusable vector. Object storage speaks the S3 API, so development runs MinIO and production substitutes a managed service with no code change. The **commodity-hardware target** is the binding constraint – the system must run on a single workstation with one consumer accelerator, at the capital cost stated in Table 5.2 – which is why the entire model stack has a stub mode in which every component degrades to a deterministic function and **reports that it did so**. A run in which every component degraded still reports the engine it *intended* to use, so the degraded-components list is the only signal distinguishing a real result from a heuristic one.

5.1.3 Communication Standards

For **transport security**, TLS terminates at the gateway, datastores and inference servers sit on private subnets, and only the gateway is internet-facing; in the cluster deployment a network policy enforces that boundary rather than relying on topology. For **message transport**, Redis Streams with consumer groups provides at-least-once delivery for the MVP tier and the production tier migrates to a partitioned log. Every stream and consumer-group name is defined in exactly one module, and a test asserts that the autoscaler manifests name the same streams and groups the workers create – a rule that exists because a scaler pointed at a group no worker creates reports no backlog, so the one stage where scaling changes cost never scaled while the manifests looked correct.

Tool transport is the Model Context Protocol: agents reach data only through MCP servers over streamable HTTP [38]. Adopting a protocol rather than writing bespoke glue is what makes a new analytical capability a tool registration plus one line in an allowlist, and the protocol's own security literature is taken seriously here – because tool results carry third-party text verbatim, everything a model is shown is wrapped in untrusted-data delimiters with forged-delimiter neutralisation [43]. **Data residency** is itself a communication constraint, because the cloud backend sends prompt content – real people's comments – to a third party. It is therefore treated as a policy rather than a performance setting: the decision is resolved where the tenant is known, stamped into the job envelope so a later global toggle cannot re-route content already in flight, re-resolved on resume, and **fails closed** when the policy table cannot be read.

5.2 Impact on Society, Environment and Sustainability

5.2.1 Impact on Life

The direct beneficiaries are the analysts who currently read comment threads by hand. On the largest thread in this corpus that means 2,857 stored comments against a platform-reported 9,224; no analyst reads that, so in practice a human reads the top twenty and generalises. The system does not remove the human and is designed on the assumption that it should not: it reports coverage, it reports what produced each label, and it shows eight verdicts side by side precisely so a human can see where the models disagree and look there.

There is a second, less obvious effect on the people whose comments are analysed. A system that labels a comment negative and stops there attributes an emotion to a person. A system that separates document sentiment from stance toward a named entity, and that says **uncertain** when no model read a comment rather than defaulting it to neutral, makes fewer claims about people than the alternative. The effect is modest, and it is the correct default when the subjects have not consented to the analysis.

5.2.2 Impact on Society and Environment

The corpus we built this system on is political content from Bangladesh, and the technology must be assessed as dual-use: the same capability that lets a newsroom see how a policy announcement was received lets an actor identify who is criticising them. Our response is to make the editorial choice visible – the watchlist is a **stated bias model** in a versioned file with a named owner per entry, and stance lives in a separate field so it cannot be folded into a sentiment total.

The environmental argument is the same as the cost argument and is measurable in the same units: every large-model call avoided is accelerator-seconds not spent. Under the shipped configuration the corpus run costs 529 calls against 813 for the keyword baseline, and the caps deliberately left disabled would reduce the dominant comment lane by more than an order of magnitude on large threads – a 2,857-comment thread costs 115 stance batches uncapped and four with the cap set to one hundred. Set against this, the eight-labeller ensemble spends roughly 0.92 seconds of CPU per comment, which is the cost of the transparency it buys.

5.2.3 Ethical Aspects

We took five ethical positions deliberately, and gave each a corresponding mechanism in the code, which is the only form in which an ethical position is checkable.

- E1. **Personal data is handled as personal data.** The system encrypts at rest and in transit, scopes every read by tenant, supports per-tenant retention and deletion, and – the mechanism that makes the rest enforceable – takes tenant identity from a stored row rather than from anything a client sends.
- E2. **A privacy commitment must survive a database outage.** The tenant policy check originally returned permissively on any database error, which permitted egress precisely when the policy could not be verified; it now returns a service error, because a privacy guarantee that evaporates when the database hiccups is not a guarantee.
- E3. **The bias in the labels is stated, not hidden.** A file declaring that support for a named entity is positive encodes a political stance – legitimate for a monitoring product and indefensible presented as neutral analysis, so it is labelled as the former throughout.
- E4. **The system does not claim more than it did.** Provenance on every label; an explicit degraded-components list; a stub flag on every stored vector; an explicit vision status where only one value licenses a claim about an image. Table 4.6 is the same commitment at the level of the whole project.
- E5. **Untrusted input is treated as untrusted.** Comment text is attacker-controlled: it is escaped before rendering, wrapped in data delimiters before a model sees it, and never permitted to alter a system instruction. This is risk reduction and is described as such, because no prompt-level defence can promise resistance to prompt injection [43].

One ethical limitation is stated plainly rather than left implicit. The comments we analysed were published publicly, but their authors did not consent to this analysis. Our mitigations – aggregate reporting, tenant scoping, retention controls, and the refusal to attribute a label to a comment no model read – reduce the harm; they do not resolve the underlying question, which is a policy matter for whoever deploys the system rather than an engineering one.

5.2.4 Sustainability Plan

Technical sustainability rests on the property that made the audit passes productive: a cross-component rename breaks a test rather than a feature. Stream names, consumer groups and label vocabularies each live in exactly one module, the deployment manifests are asserted against those modules, and every repaired defect has a regression test. **Operational sustainability** rests on the backend being pluggable: an operator with accelerators runs everything in-house at zero marginal token cost and one without them runs the same code against a metered API, and because the switch is resolved per call this is a configuration decision rather than a migration. **Economic sustainability** is addressed in Section 5.3, and **scientific sustainability** is the corpus – 10,272 real code-mixed comments with crowd reaction signals, a stratified sampler and a scorer, ready for the annotation effort that would turn this engineering result into a benchmark result.

5.3 Project Management and Financial Analysis

We managed the project through the iterative milestone approach of Section 3.3, organised around phase exit criteria rather than dates alone, with a dedicated audit phase standing in for the second reader a team would have provided. We prioritised tasks by one principle: get the pipeline and its router correct and measurable first, build the capability layers on top, and only then spend a fifth of the schedule on audit and evaluation – an allocation that was not in our original plan and is our largest management decision.

Every software component is open source and inference ran locally, so we paid no licence or per-token fees. It is not, however, a zero-cost project: this is **accelerator-bound, storage-heavy work**, and the hardware is the dominant line in its budget. We therefore report the hardware in Table 5.1 as capital rather than writing it off as already owned, with the power and connectivity our runs consumed, each set against the alternative we rejected. Figures are in US dollars at market prices; the hardware line is a purchase price, we computed power and connectivity from measured draw and duration, and every alternate-column figure is an indicative market rate rather than a quotation.

Table 5.1: Cost Analysis of Resources Used in Project Development.

Resource	Actual approach and cost	Alternate approach and cost
Workstation and accelerator – the largest line	An Intel Core i7-11800H (8 cores, 16 threads), 32 GB of memory, a 512 GB NVMe drive, an NVIDIA RTX 3050 with 4 GB of accelerator memory and an integrated GPU: the machine on which we took every measurement in Chapter 4. US\$1,150 of capital , owned before the project began and charged here at about \$32 per month over a three-year life, so roughly \$270 falls inside the 37-week schedule.	Renting an equivalent cloud GPU instance for the same hours: about \$250–600 . A 24 GB workstation card, which would have removed the CPU-versus-accelerator split of Section 5.1.2 entirely, costs about \$1,600–2,000 outright and we deliberately did not buy one – the split is the cheaper answer.
Electricity	The workstation draws roughly 110 W at the wall under sustained inference; across about 900 hours of development and measurement that is close to 100 kWh, or about \$8–12 at the domestic tariff.	Bundled into a rented instance's hourly rate and therefore not separable, which is itself a reason the rented figure looks smaller than it is.
Connectivity	Domestic broadband, shared with other use, carrying the upstream pulls and roughly 20 GB of model weights: about \$90 apportioned over the schedule.	The same requirement on every approach, and larger on a metered backend, which sends every prompt over it.
LLM inference	Ollama serving four quantised open-weight models on the owned card: \$0 marginal , because the capital is already committed and there is no per-token charge.	Metered frontier API: about \$45–90 for the measured corpus runs. Zero marginal token cost is what makes a coverage decision not also a budget decision, and no comment text leaves the host.
Classifier fleet	Seven open-weight sentiment heads batched on CPU, which is why the accelerator is not the bottleneck: \$0 .	Commercial multilingual sentiment API: about \$120–400 per corpus run, and Bangla-specific fine-tunes are only available as open weights at all.
Datastores and software stack	PostgreSQL with pgvector, ClickHouse, Redis Streams, MinIO, Vite, Prometheus and Grafana, all open source and self-hosted in Docker on the same host: \$0 in licence fees , at the cost of the memory and disk the capital line already pays for.	Managed Postgres, a hosted vector database, a managed warehouse, a hosted queue, cloud object storage, managed front-end hosting and a hosted observability platform: about \$225–695 per month .
Total	About \$370 attributable to the 37-week schedule, against \$1,150 of hardware capital committed before it. We were cheap because the hardware was already there, not because the work is free of hardware.	Fully managed equivalent: about \$2,300–6,900 over the same period, and no owned asset at the end of it.

Our development machine is the smallest configuration this system runs on, and small enough to be the binding constraint: a 4 GB accelerator already serving a seven-billion-parameter model leaves about 285 MB free, which is why the classifier fleet runs on CPU at all. A deployment serving an operator rather than a student needs server-class hardware, and Table 5.2 states what each tier of Section 3.2.10 requires – indicative market prices for the class of machine, and the capital the self-hosted envelopes below amortise.

Table 5.2: Server-class hardware required by each deployment tier, and its capital cost.

Tier	Hardware it requires	Indicative capital	Rented or metered equivalent
Development – used here	One workstation: 8 cores, 32 GB of memory, 512 GB NVMe and one 4 GB accelerator, with all four datastores and eleven processes on the same host	about \$1,150	A cloud GPU instance at roughly \$0.30–0.70 per hour
MVP – Docker Compose	One or two rack hosts: 16 to 32 cores, 64 to 128 GB of memory, 2 TB of NVMe, and a single 24 to 48 GB accelerator where the local backend is selected	about \$5,000–10,000	\$170–800 per month self-hosted against \$20–440 metered
Production – Kubernetes, two GPU node pools	A dual-socket GPU server for the large-model pool – two 64-core CPUs, 1 TB of system memory and four 48 GB accelerators, giving roughly 192 GB of accelerator memory; two CPU-only worker nodes of 2 x 64 cores and 512 GB for the Stage-1 classifier fleet, which is where the comment lane's volume actually lands; two data-tier nodes for PostgreSQL and ClickHouse with replicas and NVMe arrays; three Kafka brokers; an object-storage node; plus 100 GbE switching, rack and uninterruptible power	about \$120,000–220,000	\$4,500–11,000 per month self-hosted against \$880–4,300 metered
Enterprise – DGX-class, election-scale	One or more NVIDIA DGX systems as the inference tier: a DGX H100 carries eight H100 accelerators of 80 GB each for 640 GB of aggregate accelerator memory , two 56-core Xeon Platinum CPUs, 2 TB of system memory and 30 TB of local NVMe, and draws about 10 kW. Nodes are joined by an InfiniBand fabric over a shared all-flash storage tier, with the CPU worker pools, data tier and network all duplicated for redundancy – one DGX at the low end, a four-node DGX BasePOD at the high one	about \$400,000–1,500,000	\$16,000–60,000 per month self-hosted against \$6,250–35,600 metered

Three conclusions follow, and the first is uncomfortable for the self-hosted position we otherwise argue for. **Owned hardware does not pay for itself at any tier in this comparison:** once the capital above is amortised into the monthly envelope, the metered path is cheaper at both ends of every range, and the gap widens rather than closes with scale. What owned hardware buys instead is the property no price can substitute for – a **privacy-locked tenant's comment text never leaves the cluster**, which the enforced backend pin of Section 3.2.5 makes a hard guarantee rather than a setting. Second, the capital is dominated by the accelerator tier and the

data tier **together**, not by the accelerator alone, because the comment lane's volume falls on storage, memory and CPU as much as on inference – the same asymmetry the routing measurement of Section 4.2.4 found in the token bill.

Third, the DGX class changes what the software may assume, not merely how fast it runs. With 640 GB of accelerator memory resident, the seven classifier heads and the large model fit **at the same time**, removing the CPU-versus-accelerator split that Section 5.1.2 adopts only because a 4 GB card leaves 285 MB free. The comment lane can then be batched at a depth the development tier cannot express, and the comment cap of Section 3.2.2 – the largest cost lever – becomes a quality decision rather than a capacity one. Nothing in the code changes: the roster is configuration, and the split exists to survive small hardware rather than to prefer it.

The figure a prospective operator actually asks about is the monthly cost of running the system, and there the same self-hosted-against-metered choice reappears with a different answer at each volume. At the MVP tier the monthly envelope runs from roughly \$170 to \$800 self-hosted against \$20 to \$440 metered; at the Production tier from \$4,500 to \$11,000 against \$880 to \$4,300; and at the DGX-based Enterprise tier from \$16,000 to \$60,000 against \$6,250 to \$35,600. The metered path is cheaper at both ends of every range, and at the MVP tier dramatically so, because it removes the accelerator line entirely. For a deployment offering this as a service the natural model is a per-tenant subscription tiered on threads analysed per month, with the coverage knobs as the tier boundary because coverage is what drives cost. A **Community** plan would be free and self-hosted for single analysts, civic groups and academic use up to about a thousand threads a month, carrying the full pipeline, the eight-labeller ensemble, keyword and semantic search, ungrounded reports and one watchlist; a **Standard** plan, metered on threads analysed with the comment cap as the boundary, would add grounded reports, the full agentic insight layer and multiple watchlists for newsrooms and brand agencies up to about ten thousand threads; and an **Enterprise** plan, priced custom with owned capacity for the steady base and metered inference for spikes, would add the enforced data-residency lock, cluster deployment and a contracted service level for election-scale monitoring above a hundred thousand threads.

The system already meters what such a model needs – token and call counts per backend, per model and per lane, with per-post pipeline spend isolated from per-question chat and agent spend. Two further revenue lines follow from the architecture: the analytical layer itself, since reports and briefings scale with questions asked rather than corpus size, and the data-residency guarantee, since for a tenant that cannot let comment text leave its infrastructure the privacy lock of Section 3.2.5 is a precondition rather than a feature.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

Table 5.3 maps our project onto the seven complex-engineering-problem attributes. The problem draws on three research literatures at once (EP1), resolves irreducible conflicts between cost, coverage, privacy and latency by exposing them as runtime knobs (EP2), and required instrumenting and running the system rather than reading it to establish that four of six routing rules were inert (EP3, EP4). Its stakeholders – analyst, operator, privacy-locked tenant, platform owner and non-consenting comment authors – want incompatible things (EP6), and its five stages, three datastores, seven model roles and nine agents are connected by contracts whose violation is silent (EP7). EP5 is blank because no single code governs a system of this kind.

Table 5.3: Mapping with Complex Engineering Problem.

EP1	EP2	EP3	EP4	EP5	EP6	EP7
Depth of Knowledge Required	Range of Conflicting Requirements	Depth of Analysis Required	Familiarity of Issues	Extent of Applicable Codes	Extent of Stakeholder Involvement	Interdependence
✓	✓	✓	✓		✓	✓

EP1 requires a further mapping onto the knowledge profile, which Table 5.4 gives. Seven of the eight categories are engaged: vector similarity and the retrieval measures (K2); durable queues and horizontal partitioning (K3); transformer encoders, code-mixed processing and prompt-injection threat models (K4); the design decisions of Table 3.3 (K5); containerised deployment and six audited regression passes (K6); the dual-use and consent assessment of Section 5.2 (K7); and the literatures of Section 2.2 (K8). K1 is blank because the system has no physical-science component.

Table 5.4: Mapping with Knowledge Profile.

K1	K2	K3	K4	K5	K6	K7	K8
Natural Sciences	Mathematics	Engineering Fundamentals	Specialist Knowledge	Engineering Design	Engineering Practice	Comprehension	Research Literature
	✓	✓	✓	✓	✓	✓	✓

5.4.2 Engineering Activities

Table 5.5 maps our project onto the five complex-engineering-activity attributes, and all five apply: we budget seven model roles, four datastores, five worker pools and three tool servers against one accelerator (EA1); reconciles conflicting requirements from five stakeholder groups (EA2); contributes watchlist-driven target stance for code-mixed Bangla at zero additional inference cost, an abstention protocol, and the measured container-routing finding (EA3); carries dual-use, energy and consent consequences (EA4); and works where the dominant failure mode is silent degradation rather than exception (EA5).

Table 5.5: Mapping with Complex Engineering Activities.

EA1	EA2	EA3	EA4	EA5
Range of Resources	Level of Interaction	Innovation	Consequences for Society and Environment	Familiarity
✓	✓	✓	✓	✓

We state our innovation claim narrowly on purpose. Target-dependent stance detection is not novel – it is established for English [13], [14], [15], [16] – and doing it over an operator-supplied entity list is an engineering framing rather than a research result. **The defensible claim is doing it on code-mixed Bangla, where the same entity is written in three scripts and the matcher is the hard part.** The line that survives scrutiny is *novel application and engineering in an under-resourced language setting, not a novel method.*

5.5 Summary

The system complies with an assembled rather than a single standard, and Section 5.1 records each choice with the alternative it displaced. Its societal impact is genuinely dual-use, and our response is to make the editorial component of its labels visible rather than claim a neutrality it does not have. Our financial analysis reports the hardware as capital rather than writing it off: about \$370 of power, connectivity and amortised workstation falls inside the schedule against \$1,150 of accelerator-bearing hardware committed before it, and a Production deployment needs \$45,000 to \$72,000 of server-class capital that no monthly envelope makes cheaper than the metered path – owned hardware buys data residency, not savings. Against the complex-engineering framework we engage six of the seven problem attributes, seven of the eight knowledge-profile categories and all five engineering activities.

Conclusion

This chapter summarises what we built and what we learned, states our limitations without softening them, and sets out the work that would close each one, in the order that would produce the most value.

6.1 Summary

We asked a narrow, testable question: given a post together with its comment thread, in a setting mixing Bangla script, English and romanized Banglish, can a system decide *per unit of work* how much intelligence that unit deserves, spend the expensive model only where cheap models cannot answer, and prove afterwards from its own counters where the money went and what produced every label?

The answer is yes, and not in the way we originally believed. We run cheap analysis on every post and every comment; a six-gate router decides post-level spend and a separate step decides which comments the expensive labellers read – two decisions rather than one, and the structural change that made the cost behaviour measurable. We raised per-comment sentiment to eight labellers with explicit abstention, added watchlist stance that answers whom a comment opposes at zero additional model calls, and built nine agents that reason over the system's own output through eighteen typed tools. Measurement then produced the key outcome: **the gate behaves exactly as designed – 16 percent of posts route against 74 percent for a keyword baseline – yet governs only 30 percent of large-model calls, because between 85 and 96 percent of calls are comment-level.** Hybrid retrieval reaches recall@10 of 0.8750 where hash-seeded vectors score 0.1875, and the whole system verifies as one pass-or-fail chain in seventy-three seconds.

Our most transferable outcome is not any of those numbers but the discipline that produced them. Six independent audit passes established that the consequential failures in a system of this shape are not incorrect answers but **silent successes** – a gate reporting plausible behaviour while four of its six rules were inert, a retrieval arm matching zero rows while a passing test covered it. Named reader functions, single-source constants, provenance on every value, abstention instead of a default vote, and an evidence class on every published claim are what we would carry to the next system.

6.2 Limitation

Three limitations bound our evidence base. Our corpus is small and its comment sample biased: fifty posts in a three-week window on one platform, whose 10,272 comments are an engagement-ordered 3.75 percent slice of the 274,126 reported, so every thread-level aggregate describes *the most-engaged comments per post*. The multimodal claim is unexercised, because the image path [28], [29] is implemented but its 69 references are object-storage keys whose objects are absent. And retrieval quality rests on 32 machine-derived queries with no human relevance judgement, which favours literal matching and leaves the decisive stratified bucket at eight.

Three further capabilities carry no measurement. Horizontal scaling, queue-depth autoscaling and the Kubernetes deployment are test-asserted but never timed in a cluster, so every throughput statement is a design property. The agent runner is hardened against six observed failure classes, each pinned by a regression test, but groundedness and citation accuracy carry no scores. Labelling accuracy is out of scope by design: our gold file holds zero adjudicated labels, because seeding them from a model here and scoring that model would measure agreement with itself, so we quote no macro-F1 or calibration curve [48].

We record three narrower constraints here rather than leaving them in a screenshot: the per-analysis event stream has a five-minute ceiling, after which the Trace tab reports a failure while the pipeline finishes correctly; the monitoring stack provisions no Grafana dashboards; and the prompt-injection hardening is risk reduction rather than a solution [43]. All nine are carried at their evidence class in Table 4.6.

6.3 Future Work

One item dominates our future work, because most of the others depend on it: **adjudicating the gold set**. Two annotators would label a five-hundred-comment overlap and report Cohen's kappa [49], then label to two or three thousand comments stratified by language bucket and kind, ruling explicitly on emoji-only sarcasm [17], political epithets and mid-sentence code-switching. The work would then become benchmarking rather than building: we would score the seven heads, the LLM stance pass and the Stage-1 baseline, reporting macro-F1 **per language bucket** with the cache disabled and provenance on every row, and fine-tune the weakest heads with a parameter-efficient adapter [27]. This is the step that would take our work from an engineering deliverable to an empirical one.

Four items follow from those labels. Sweeping the router's confidence threshold would turn the cascade into a measured trade-off, plotting macro-F1 against cost with the comment cap as a second curve. The watchlist needs validating on two metrics reported apart – mention recall, because a missed alias is invisible, and stance agreement on correctly matched mentions. The routing decision should then be learned rather than hand-tuned [31], and the image modality made real or its weights deleted, since a weight that has never been anything but zero is not defensible either way.

The rest needs no annotation. We would score the agent layer against an analyst question set with reference answers, measuring groundedness claim by claim, citation accuracy and tool-use correctness with a judge model and human review on a sample [44], [45], [47] and the retrieval measures of [46]. A multi-replica run against a ten-thousand-thread batch would convert NFR1 into a measurement; the trace view should resubscribe at the five-minute ceiling; the monitoring stack should ship dashboards; and the access-control surface deserves review by someone who did not write it. Together, an estimated three to four weeks dominated by annotation.

References

References follow the IEEE citation style. Every entry carries a Digital Object Identifier that was resolved against the Crossref or DataCite registry at the time of writing.

- [1] A. Bhattacharjee, T. Hasan, W. Ahmad, K. S. Mubasshir, M. S. Islam, A. Iqbal, M. S. Rahman, and R. Shahriyar, "BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla," in *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022, pp. 1318-1327, doi: 10.18653/v1/2022.findings-naacl.98.
- [2] K. I. Islam, S. Kar, M. S. Islam, and M. R. Amin, "SentNoB: A dataset for analysing sentiment on noisy Bangla texts," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 3265-3271, doi: 10.18653/v1/2021.findings-emnlp.278.
- [3] A. Hassan, M. R. Amin, A. K. A. Azad, and N. Mohammed, "Sentiment analysis on Bangla and romanized Bangla text using deep recurrent models," in *Proc. 2016 Int. Workshop on Computational Intelligence (IWCI)*, 2016, pp. 51-56, doi: 10.1109/IWCI.2016.7860338.
- [4] S. Sazzed, "Abusive content detection in transliterated Bengali-English social media corpus," in *Proc. Fifth Workshop on Computational Approaches to Linguistic Code-Switching*, 2021, pp. 125-130, doi: 10.18653/v1/2021.calcs-1.16.
- [5] N. Romim, M. Ahmed, H. Talukder, and M. Saiful Islam, "Hate speech detection in the Bengali language: A dataset and its baseline evaluation," in *Proc. Int. Joint Conf. on Advances in Computational Intelligence, Algorithms for Intelligent Systems*. Singapore: Springer, 2021, pp. 457-468, doi: 10.1007/978-981-16-0586-4_37.
- [6] M. A. Nezami and M. N. Huda, "Public sentiment identification in social media for code-mixed English-Bangla and Banglish text analysis with machine learning," in *Proc. Sixth Int. Conf. on Image, Video Processing, and Artificial Intelligence (IVPAI 2024)*, SPIE, 2024, doi: 10.1117/12.3047050.
- [7] S. Saha and A. Nanda, "BanglaNLP at BLP-2023 Task 2: Benchmarking different transformer models for sentiment analysis of Bangla social media posts," in *Proc. First Workshop on Bangla Language Processing (BLP-2023)*, 2023, pp. 266-272, doi: 10.18653/v1/2023.banglalp-1.34.
- [8] A. Chakma and M. Hasan, "LowResource at BLP-2023 Task 2: Leveraging BanglaBert for low-resource sentiment analysis of Bangla language," in *Proc. First Workshop on Bangla Language Processing (BLP-2023)*, 2023, pp. 347-353, doi: 10.18653/v1/2023.banglalp-1.47.
- [9] F. Alam, A. Hasan, T. Alam, A. Khan, J. Tajrin, N. Khan, and S. A. Chowdhury, "A review of Bangla natural language processing tasks and the utility of transformer models," arXiv:2107.03844, 2021, doi: 10.48550/arXiv.2107.03844.
- [10] P. Patwa, G. Aguilar, S. Kar, S. Pandey, S. PYKL, B. Gamback, T. Chakraborty, T. Solorio, and A. Das, "SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets," in *Proc. Fourteenth Workshop on Semantic Evaluation*, 2020, pp. 774-790, doi: 10.18653/v1/2020.semeval-1.100.
- [11] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," arXiv:1904.00784, 2019, doi: 10.48550/arXiv.1904.00784.
- [12] M. Wankhade, A. C. S. Rao, and C. Kulkarni, "A survey on sentiment analysis methods, applications, and challenges," *Artificial Intelligence Review*, vol. 55, no. 7, pp. 5731-5780, 2022, doi: 10.1007/s10462-022-10144-1.
- [13] D. Kucuk and F. Can, "Stance detection: A survey," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1-37, 2021, doi: 10.1145/3369026.

- [14] S. Mohammad, S. Kiritchenko, P. Sobhani, X. Zhu, and C. Cherry, "SemEval-2016 Task 6: Detecting stance in tweets," in *Proc. 10th Int. Workshop on Semantic Evaluation (SemEval-2016)*, 2016, pp. 31-41, doi: 10.18653/v1/S16-1003.
- [15] M. Pontiki, D. Galanis, J. Pavlopoulos, H. Papageorgiou, I. Androutsopoulos, and S. Manandhar, "SemEval-2014 Task 4: Aspect based sentiment analysis," in *Proc. 8th Int. Workshop on Semantic Evaluation (SemEval 2014)*, 2014, pp. 27-35, doi: 10.3115/v1/S14-2004.
- [16] L. Dong, F. Wei, C. Tan, D. Tang, M. Zhou, and K. Xu, "Adaptive recursive neural network for target-dependent Twitter sentiment classification," in *Proc. 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2014, pp. 49-54, doi: 10.3115/v1/P14-2009.
- [17] P. Kralj Novak, J. Smailovic, B. Sluban, and I. Mozetic, "Sentiment of emojis," *PLOS ONE*, vol. 10, no. 12, e0144296, 2015, doi: 10.1371/journal.pone.0144296.
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv:1706.03762, 2017, doi: 10.48550/arXiv.1706.03762.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.
- [20] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzman, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440-8451, doi: 10.18653/v1/2020.acl-main.747.
- [21] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. 15th Conf. of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2017, pp. 427-431, doi: 10.18653/v1/E17-2068.
- [22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980-3990, doi: 10.18653/v1/D19-1410.
- [23] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4512-4525, doi: 10.18653/v1/2020.emnlp-main.365.
- [24] B. Warner, A. Chaffin, B. Clavie, O. Weller, O. Hallstrom, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," arXiv:2412.13663, 2024, doi: 10.48550/arXiv.2412.13663.
- [25] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proc. 2024 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 5364-5376, doi: 10.18653/v1/2024.naacl-long.300.
- [26] W. Yin, J. Hay, and D. Roth, "Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3912-3921, doi: 10.18653/v1/D19-1404.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021, doi: 10.48550/arXiv.2106.09685.

- [28] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," arXiv:2103.00020, 2021, doi: 10.48550/arXiv.2103.00020.
- [29] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proc. 2023 IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023, pp. 11941-11952, doi: 10.1109/ICCV51070.2023.01100.
- [30] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023, doi: 10.48550/arXiv.2305.05176.
- [31] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," arXiv:2406.18665, 2024, doi: 10.48550/arXiv.2406.18665.
- [32] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," arXiv:2404.14618, 2024, doi: 10.48550/arXiv.2404.14618.
- [33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611-626, doi: 10.1145/3600006.3613165.
- [34] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," arXiv:2005.11401, 2020, doi: 10.48550/arXiv.2005.11401.
- [35] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023, doi: 10.48550/arXiv.2312.10997.
- [36] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022, doi: 10.48550/arXiv.2210.03629.
- [37] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," arXiv:2302.04761, 2023, doi: 10.48550/arXiv.2302.04761.
- [38] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," arXiv:2503.23278, 2025, doi: 10.48550/arXiv.2503.23278.
- [39] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758-759, doi: 10.1145/1571941.1572114.
- [40] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020, doi: 10.1109/TPAMI.2018.2889473.
- [41] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422-446, 2002, doi: 10.1145/582415.582418.
- [42] F. Cuconasu, G. Trappolini, F. Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonello, and F. Silvestri, "The power of noise: Redefining retrieval for RAG systems," in *Proc. 47th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2024, pp. 719-729, doi: 10.1145/3626772.3657834.
- [43] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th*

ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

- [44] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," arXiv:2306.05685, 2023, doi: 10.48550/arXiv.2306.05685.
- [45] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12076-12100, doi: 10.18653/v1/2023.emnlp-main.741.
- [46] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv:2309.15217, 2023, doi: 10.48550/arXiv.2309.15217.
- [47] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1-45, 2024, doi: 10.1145/3641289.
- [48] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," arXiv:1706.04599, 2017, doi: 10.48550/arXiv.1706.04599.
- [49] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104.
- [50] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*, Lecture Notes in Computer Science, vol. 1857. Berlin, Heidelberg: Springer, 2000, pp. 1-15, doi: 10.1007/3-540-45014-9_1.